

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №4 по курсу
«Операционные системы»

Группа: М8О-214Б-23

Студент: Кондратенко А.С.

Преподаватель: Бахарев В.Д. (ФИИТ)

Оценка: _____

Дата: 08.01.25

Москва, 2024

Постановка задачи

Вариант 7.

Исследовать два аллокатора памяти: необходимо реализовать два алгоритма аллокации памяти и сравнить их по следующим характеристикам:

- Фактор использования
- Скорость выделения блоков
- Скорость освобождения блоков
- Простота использования аллокатора

Требуется создать две динамические библиотеки, реализующие два аллокатора, соответственно. Библиотеки загружаются в память с помощью интерфейса ОС (dlopen / LoadLibrary) для работы с динамическими библиотеками. Выбор библиотеки, реализующей аллокатор, осуществляется чтением первого аргумента при запуске программы (argv[1]). Этот аргумент должен содержать путь до динамической библиотеки (относительный или абсолютный).

Если аргумент не передан или по переданному пути библиотеки не оказалось, то указатели на функции, реализующие API аллокатора ниже, должны быть присвоены функциям, которые оборачивают системный аллокатор ОС (mmap / VirtualAlloc) в этот API. Эти аварийные оберточные функции должны быть реализованы внутри программы, которая загружает динамические библиотеки (см. пример на GitHub Gist). Каждый аллокатор памяти должен иметь функции аналогичные стандартным функциям malloc, free, realloc, (опционально). Перед работой каждый аллокатор инициализируется свободными страницами памяти, выделенными стандартными средствами ядра (mmap / VirtualAlloc). Необходимо самостоятельно разработать стратегию тестирования для определения ключевых характеристик аллокаторов памяти. При тестировании нужно свести к минимуму потери точности из-за накладных расходов при измерении ключевых характеристик, описанных выше.

Списки свободных блоков (наиболее подходящее) и блоки по 2^n .

Общий метод и алгоритм решения

Использованные системные вызовы:

VirtualAlloc: Выделение блока виртуальной памяти заданного размера. Используется для получения памяти под аллокаторы и их нужды.

VirtualFree: Освобождение блока памяти, выделенного через VirtualAlloc.

LoadLibrary: Загрузка динамической библиотеки (DLL) в память процесса. Применяется для загрузки библиотек с реализациями аллокаторов.

GetProcAddress: Получение адреса экспортируемой функции из загруженной DLL. Используется для доступа к функциям API аллокаторов.

FreeLibrary: Выгрузка динамической библиотеки из памяти процесса.

Я реализовал собственный аналог функции printf (my_printf). В ней обрабатывается форматная строка, в результирующую строку подставляются переданные аргументы, и затем она

выводится в консоль (с помощью WriteConsoleA). Также я создал функцию для перевода строки в LPWSTR.

Работа программы lab44.cpp:

Программа тестирует два алгоритма аллокации памяти, реализованных в отдельных DLL.

Выбор аллокатора: при запуске проверяет аргумент командной строки. Если он есть и указывает на существующую DLL, загружает её (LoadLibrary). В противном случае использует обертки вокруг VirtualAlloc/VirtualFree.

Получение функций: из загруженной DLL (GetProcAddress) или из оберток получает указатели на функции API аллокатора (allocator_create, allocator_destroy, allocator_alloc, allocator_free, print_blocks(для иллюстрации работы аллокатора)).

Инициализация: выделяет память (VirtualAlloc) и инициализирует выбранный аллокатор (allocator_create).

Бенчмарк: многократно выделяет (allocator_alloc) и освобождает (allocator_free) блоки памяти случайных размеров, измеряя время выполнения этих операций.

Деинициализация: деинициализирует аллокатор (allocator_destroy) и выгружает DLL (FreeLibrary), если она загружалась.

Вывод результатов: выводит информацию о времени выделения и освобождения памяти, а также факторе использования памяти.

Подробное описание каждого из исследуемых алгоритмов

1. Списки свободных блоков(наиболее подходящее).

Аллокатор памяти через список свободных блоков – это метод управления динамической памятью, при котором свободные блоки памяти хранятся в виде связного списка. Этот алгоритм позволяет эффективно распределять память для различных объектов программы и освобождать её после использования. При запуске программы выделяется большой блок памяти, который будет использоваться для хранения данных. Весь этот блок помечается как свободный и добавляется в список свободных блоков.

Выделение памяти: Алгоритм находит наименьший блок, который удовлетворяет запросу. Это помогает минимизировать фрагментацию, но требует большего времени на поиск подходящего блока, чем при первом подходящем блоке. Если найденный блок больше требуемого размера, он разделяется на две части: одна часть используется для выделения памяти, а другая остаётся свободной и возвращается в список свободных блоков.

Освобождение памяти: Аллокатор памяти находит соответствующий блок в списке свободных блоков по переданному адресу. Этот блок помечен как занятый, поэтому необходимо изменить его статус на «свободный». Блоку присваивается статус «свободный», и он снова становится доступным для распределения другим запросам.

Преимущества: Простота реализации, возможность эффективного управления большими объёмами памяти.

Недостатки: Фрагментация памяти, особенно при поиске первого/наименее подходящего блока. Дополнительная нагрузка на процессор при поиске подходящего блока.

2. Блоки по 2^n .

Аллокатор памяти, который работает с блоками размером 2^n , называется аллокатором двоичных блоков или аллокатором степеней двойки. Этот алгоритм часто используется для управления динамической памятью, так как он позволяет эффективно управлять выделением и освобождением памяти без фрагментации. Память делится на блоки размера 2^n байт, где n может принимать значения от степени двойки, покрывающей размер блока, до некоторого максимального числа, определяемого архитектурой системы. Блоки хранятся в массиве списков.

Выделение памяти: Когда программе требуется память, она подбирает минимальную степень двойки, покрывающую данный объем памяти. Если в списке блоков соответствующего размера есть свободный, то он выделяется. Иначе блоки большего размера делятся пополам, и аллокатор пытается выделить один из них.

Освобождение памяти: При освобождении блока он возвращается в список свободных блоков нужного размера.

Преимущества: Простота реализации: алгоритмы выделения и освобождения памяти достаточно просты и эффективны; минимизация внешней фрагментации: поскольку все блоки имеют размер 2^n , вероятность возникновения внешней фрагментации значительно снижается.

Недостатки: Внутренняя фрагментация: может возникать внутренняя фрагментация, когда программа запрашивает меньше памяти, чем предоставляет минимальный блок 2^n .

Процесс тестирования

Для тестирования реализованных аллокаторов памяти был разработан следующий процесс:

1. Динамическая загрузка библиотек: Программа `main.c` принимает в качестве аргумента путь к динамической библиотеке (`.dll`). Используются системные вызовы `LoadLibrary` для загрузки библиотеки и `GetProcAddress` для получения указателей на функции API аллокатора (`allocator_create`, `allocator_destroy`, `allocator_alloc`, `allocator_free`).

2. Инициализация аллокатора: После загрузки библиотеки выделяется статический буфер памяти с помощью `VirtualAlloc`. Затем вызывается функция `allocator_create` из загруженной библиотеки для инициализации аллокатора в выделенном буфере.

3. Серия выделений и освобождений: Выполняется заданное количество итераций. На каждой итерации вызывается функция `allocator_alloc` для выделения блока памяти определенного размера (размеры сохранены в массиве `sz`). Выделенные блоки сохраняются в массиве `rs`.

4. Измерение времени выделения: Измеряется время, затраченное на конкретную операцию и выводится на экран.

5. Освобождение выделенных блоков: После всех выделений выполняется последовательное освобождение всех выделенных ранее блоков с помощью функции ``allocator_free``.

6. Измерение времени освобождения: Аналогично времени выделения, измеряется время, затраченное на операции освобождения.

7. Деинициализация и выгрузка: После тестирования вызываются функции ``allocator_destroy`` для деинициализации аллокатора и ``FreeLibrary`` для выгрузки динамической библиотеки.

8. Тестирование системного аллокатора: Если при запуске не был указан путь к динамической библиотеке или загрузка библиотеки не удалась, программа использует обертки вокруг системных функций ``VirtualAlloc`` и ``VirtualFree`` для имитации работы аллокатора. Это позволяет сравнить производительность пользовательских аллокаторов с системным.

Обоснование подхода тестирования

Выбранный подход к тестированию основан на следующих принципах:

Реалистичная нагрузка: Моделируется ситуация, когда приложение выполняет многократные выделения и освобождения блоков памяти различных размеров, что типично для многих программ. Использование случайных размеров выделяемых блоков приближает нагрузку к реальным условиям.

Изоляция аллокатора: Использование динамических библиотек позволяет изолировать реализацию каждого аллокатора и избежать влияния других частей программы на результаты тестирования.

Измерение ключевых характеристик: Основное внимание уделяется измерению времени выделения и освобождения памяти, поскольку это критически важные параметры производительности аллокаторов.

Сравнение с системным аллокатором: Предоставление возможности использовать системный аллокатор в качестве "резервного" варианта позволяет получить базовые показатели производительности и сравнить с ними пользовательские реализации. Это помогает оценить, насколько разработанные аллокаторы эффективнее (или менее эффективны) стандартных средств.

Минимизация накладных расходов измерения: Измерение общего времени для большого количества операций (ALLOCATIONS) позволяет усреднить случайные колебания и уменьшить влияние накладных расходов, связанных с вызовами функций измерения времени.

Результаты тестирования

Результаты тестирования представлены ниже:

Объем памяти (байты)	Список свободных блоков (аллокация, с)	Список свободных блоков (освобождение, с)	Степень двойки (аллокация, с)	Степень двойки (освобождение, с)	Системный аллокатор (аллокация, с)	Системный аллокатор (освобождение, с)
10	0.100000	0.100000	0.200000	0.100000	0.100000	0.100000
20	0.200000	0.100000	0.100000	0.100000	0.0	0.200000
100	0.200000	0.100000	0.500000	0.200000	0.0	0.200000
150	0.170000	0.0	0.200000	0.100000	0.0	0.100000
1024	0.300000	0.0	0.700000	0.700000	0.100000	0.0
80	0.400000	0.300000	0.400000	0.100000	0.0	0.0
35	0.400000	0.0	0.100000	0.0	0.100000	0.100000

32	0.700000	0.0	0.340000	0.0	0.0	0.200000
55	0.899999	0.500000	0.600000	0.100000	0.0	0.100000
2048	0.899999	0.100000	0.300000	0.850000	0.200000	0.100000
5000	0.930000	0.100000	0.200000	0.900000	0.300000	0.100000
136	0.940000	0.0	0.600000	0.200000	0.200000	0.100000
25	0.940000	0.0	0.500000	0.0	0.300000	0.100000

Анализ результатов:

Фактор использования: Алгоритм через список свободных блоков демонстрирует значительно более высокий фактор использования памяти по сравнению с аллокатором степеней двойки. Это связано с тем, что он позволяет выделять блоки более точно под размер запроса, минимизируя внутреннюю фрагментацию. Аллокатор степеней двойки, в свою очередь, выделяет блоки размером, кратным степени двойки, что приводит к большему количеству неиспользуемого пространства внутри выделенных блоков.

Скорость выделения: У алгоритма через список свободных блоков значительно более высокая скорость выделения памяти по сравнению с аллокатором степеней двойки и сопоставимая с системным аллокатором при малом количестве вызовов функции `allocator_alloc`. Это может быть связано с более простой логикой поиска и выделения блоков в данном конкретном наборе тестов и отсутствием необходимости рекурсивно делить блоки. Однако с увеличением размера списка этот алгоритм начинает проигрывать по времени (т.к. при каждом выделении просматривается весь список).

Скорость освобождения: Время освобождения у алгоритма через список свободных блоков и системного аллокатора на блоках равного размера примерно одинаково и значительно ниже, чем у аллокатора степеней двойки. Это может указывать на более эффективную реализацию процесса освобождения в этих алгоритмах в данном тесте (нужно просто добавить освободившийся блок в список без подбора степеней двойки).

Простота реализации: Если важна простота, предпочтительнее использовать аллокатор через список свободных блоков. Он легче в реализации, управлении, и оптимизации использования памяти.

Код программы

funcs_freeblocks.h

```
#ifdef DLLFREEBLOCKS_EXPORTS

#define MYLIBRARY_API __declspec(dllexport)

#else

#define MYLIBRARY_API __declspec(dllimport)

#endif


#define MAX_SIZE 1024

#include <stdlib.h>

#include <string.h>

#include <windows.h>


typedef struct Block {

    size_t free_space;

    size_t size;

    struct Block* next;

} FreeBlock;


typedef struct Allocator {

    Block* blocks;

    void* memory;

    size_t total_size;

} Allocator;


extern "C" MYLIBRARY_API void my_printf(const char* format, ...);


extern "C" MYLIBRARY_API Allocator* allocator_create(void* memory, const size_t size);
```

```
extern "C" MYLIBRARY_API void allocator_destroy(Allocator* allocator);
```

```
extern "C" MYLIBRARY_API void print(Block* block);
```

```
extern "C" MYLIBRARY_API void print_blocks(Allocator* allocator);
```

```
extern "C" MYLIBRARY_API void* allocator_alloc(Allocator* allocator, const size_t size);
```

```
extern "C" MYLIBRARY_API void allocator_free(Allocator* allocator, void* memory);
```

```
extern "C" MYLIBRARY_API void my_printf(const char* format, ...);
```

funcs_freeblocks.cpp

```
#include "pch.h"
```

```
#include "funcs.h"
```

```
extern "C" MYLIBRARY_API void my_printf(const char* format, ...) {
```

```
    va_list args;
```

```
    va_start(args, format);
```

```
    char buffer[1024];
```

```
    char* buf_ptr = buffer;
```

```
    const char* fmt_ptr = format;
```

```
    int buffer_size = sizeof(buffer);
```

```
    while (*fmt_ptr) {
```

```
        if (*fmt_ptr == '%') {
```

```
            fmt_ptr++;
```

```
            switch (*fmt_ptr) {
```

```
                case 'd': {
```

```
                    int value = va_arg(args, int);
```

```

char num_buffer[20];

char* num_ptr = num_buffer;

if (value < 0) {
    *buf_ptr++ = '-';
    value = -value;
}

do {
    *num_ptr++ = (char)((value % 10) + '0');
    value /= 10;
} while (value > 0);

while (num_ptr > num_buffer) {
    *buf_ptr++ = *--num_ptr;
}

break;
}

case 'l': {
    unsigned long long value = va_arg(args, unsigned long long);
    char num_buffer[40];
    char* num_ptr = num_buffer;
    do {
        *num_ptr++ = (char)((value % 10) + '0');
        value /= 10;
    } while (value > 0);
    while (num_ptr > num_buffer) {
        *buf_ptr++ = *--num_ptr;
    }
    break;
}

case 'k': {
    double value = va_arg(args, double);

```

```

char num_buffer[20];

char* num_ptr = num_buffer;

if (value < 0) {
    *buf_ptr++ = '-';
    value = -value;
}

int afterDot = (value - (int)value) * 100000000;
int beforeDot = (int)value;

do {
    *num_ptr++ = (char)((afterDot % 10) + '0');
    afterDot /= 10;
} while (afterDot > 0);

*num_ptr++ = '.';

do {
    *num_ptr++ = (char)((beforeDot % 10) + '0');
    beforeDot /= 10;
} while (beforeDot > 0);

while (num_ptr > num_buffer) {
    *buf_ptr++ = *--num_ptr;
}

break;
}

case 's': {
    char* str = va_arg(args, char*);
    while (*str) {

```

```

        *buf_ptr++ = *str++;
    }

    break;
}

case 'c': {
    char ch = (char)va_arg(args, int);

    *buf_ptr++ = ch;

    break;
}

case '%': {
    *buf_ptr++ = '%';

    break;
}

default:
    *buf_ptr++ = *fmt_ptr;

    break;
}

}

else {
    *buf_ptr++ = *fmt_ptr;
}

fmt_ptr++;
}

*buf_ptr = '\0';

va_end(args);

HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);

DWORD bytesWritten;

WriteConsoleA(hConsole, buffer, (DWORD)(buf_ptr - buffer), &bytesWritten, NULL);

```

```
}
```

```
extern "C" MYLIBRARY_API Allocator* allocator_create(void* memory, const size_t size) {
```

```
    Allocator* allocator = (Allocator*)memory;
```

```
    if (size < sizeof(Allocator))
```

```
        return NULL;
```

```
    allocator->memory = (char*)memory + sizeof(Allocator);
```

```
    allocator->total_size = size - sizeof(Allocator);
```

```
    allocator->blocks = (Block*)allocator->memory;
```

```
    allocator->blocks->free_space = allocator->total_size - sizeof(Block);
```

```
    allocator->blocks->size = 0;
```

```
    allocator->blocks->next = NULL;
```

```
    my_printf("Allocator size: %d Block size: %d\n", sizeof(Allocator), sizeof(Block));
```

```
    my_printf("Allocator was created, size (without 1 block) = %d\n", allocator->blocks->free_space);
```

```
    my_printf("Beginning %d\n", (char*)allocator->memory - (char*)memory);
```

```
    return allocator;
```

```
}
```

```
extern "C" MYLIBRARY_API void allocator_destroy(Allocator* allocator) {
```

```
    allocator->blocks = NULL;
```

```
    my_printf("Allocator was destroyed\n");
```

```
}
```

```
extern "C" MYLIBRARY_API void print(Block* block) {
```

```

if (!block) {

    my_printf("Block: NULL\n");

    return;

}

my_printf("Block: free: %d size: %d\n", block->free_space, block->size);

}

extern "C" MYLIBRARY_API void print_blocks(Allocator* allocator) {

    my_printf("Blocks:-----\n");

    Block* cur_block = allocator->blocks;

    while (cur_block) {

        print(cur_block);

        cur_block = cur_block->next;

    }

    my_printf("-----\n");

}

extern "C" MYLIBRARY_API void* allocator_alloc(Allocator* allocator, const size_t size) {

    my_printf("Start allocationg, size = %d\n", size);

    Block* cur_block = allocator->blocks;

    Block* prev = NULL;

    size_t size_with_block = size + sizeof(Block);

    my_printf("Size with block: %d\n", size_with_block);

    Block* best_block = NULL;

    while (cur_block) {

        if (cur_block->free_space >= size_with_block && (best_block == NULL || cur_block->free_space
        < best_block->free_space)) {

            best_block = cur_block;


```

```

    }

    prev = cur_block;

    cur_block = cur_block->next;
}

my_printf("Best: ");
print(best_block);

if (best_block == NULL)

    return NULL;

if (best_block->size == 0) {

    best_block->size = size;

    best_block->free_space -= size;

    return (void*)((char*)best_block + sizeof(Block));
}

if (best_block->free_space > size_with_block + sizeof(Block)) {

    my_printf("Sharing (%d > %d)\n", best_block->free_space, size_with_block + sizeof(Block));

    Block* new_block = (Block*)((char*)best_block + best_block->size + sizeof(Block));

    new_block->free_space = best_block->free_space - size_with_block;

    new_block->next = best_block->next;

    new_block->size = size;

    my_printf("New block: ");

    print(new_block);

```

```

best_block->free_space = 0;

if (prev)
    prev->next = new_block;
else
    allocator->blocks = new_block;

return (void*)((char*)new_block + sizeof(Block));

}

else {
    my_printf("Deleting\n");
    best_block->free_space = 0;
    if (prev)
        prev->next = best_block->next;
    else
        allocator->blocks = best_block->next;

    return (void*)((char*)best_block + sizeof(Block));
}

}

extern "C" MYLIBRARY_API void allocator_free(Allocator* allocator, void* memory) {
    if (!memory)
        return;

    Block* block = (Block*)((char*)memory - sizeof(Block));
    my_printf("Block to free: ");
    print(block);
}

```

```
    block->free_space += block->size;

    block->size = 0;
}
```

funcs_pow2.h

```
#ifndef DLLPOW2_EXPORTS

#define MYLIBRARY_API __declspec(dllexport)

#else

#define MYLIBRARY_API __declspec(dllimport)

#endif


#include <stdlib.h>

#include <math.h>

#include <string.h>

#include <stdarg.h>

#include <windows.h>


#define N_BLOCKS 10

#define NUM_LISTS 11      // Количество списков (от 0 до 10)


typedef struct Block {

    struct Block* next;

    struct Block* prev;

    size_t size;

} Block;


typedef struct Allocator {

    Block* freeLists[NUM_LISTS];

    void* memory;

    size_t total_size;
```

```
    size_t start_extent;

} Allocator;


extern "C" MYLIBRARY_API void my_printf(const char* format, ...);


extern "C" MYLIBRARY_API int power(int base, int exp);


extern "C" MYLIBRARY_API void print_blocks(Allocator* a);


extern "C" MYLIBRARY_API Allocator* allocator_create(void* mem, size_t size);


extern "C" MYLIBRARY_API void split_block(Allocator* allocator, Block* block);


extern "C" MYLIBRARY_API void* allocator_alloc(Allocator* allocator, size_t size);


extern "C" MYLIBRARY_API void allocator_free(Allocator* allocator, void* ptr);


extern "C" MYLIBRARY_API void allocator_destroy(Allocator* allocator);
```

funcs_pow2.cpp

```
#include "pch.h"

#include "funcs.h"


extern "C" MYLIBRARY_API void my_printf(const char* format, ...) {

    va_list args;

    va_start(args, format);


    char buffer[1024];

    char* buf_ptr = buffer;

    const char* fmt_ptr = format;

    int buffer_size = sizeof(buffer);
```

```

while (*fmt_ptr) {
    if (*fmt_ptr == '%') {
        fmt_ptr++;
        switch (*fmt_ptr) {
        case 'd': {
            int value = va_arg(args, int);

            char num_buffer[20];

            char* num_ptr = num_buffer;

            if (value < 0) {
                *buf_ptr++ = '-';

                value = -value;
            }

            do {
                *num_ptr++ = (char)((value % 10) + '0');

                value /= 10;
            } while (value > 0);

            while (num_ptr > num_buffer) {
                *buf_ptr++ = *--num_ptr;
            }

            break;
        }

        case 'l': {
            unsigned long long value = va_arg(args, unsigned long long);

            char num_buffer[40];

            char* num_ptr = num_buffer;

            do {
                *num_ptr++ = (char)((value % 10) + '0');

                value /= 10;
            } while (value > 0);

```

```

while (num_ptr > num_buffer) {

    *buf_ptr++ = *--num_ptr;

}

break;

}

case 'k': {

    double value = va_arg(args, double);

    char num_buffer[20];

    char* num_ptr = num_buffer;

    if (value < 0) {

        *buf_ptr++ = '-';

        value = -value;

    }


    int afterDot = (value - (int)value) * 100000000;

    int beforeDot = (int)value;


    do {

        *num_ptr++ = (char)((afterDot % 10) + '0');

        afterDot /= 10;

    } while (afterDot > 0);


    *num_ptr++ = '.';


    do {

        *num_ptr++ = (char)((beforeDot % 10) + '0');

        beforeDot /= 10;

    } while (beforeDot > 0);


    while (num_ptr > num_buffer) {

```

```

        *buf_ptr++ = *--num_ptr;
    }

    break;
}

case 's': {

    char* str = va_arg(args, char*);

    while (*str) {

        *buf_ptr++ = *str++;

    }

    break;
}

case 'c': {

    char ch = (char)va_arg(args, int);

    *buf_ptr++ = ch;

    break;
}

case '%': {

    *buf_ptr++ = '%';

    break;
}

default:

    *buf_ptr++ = *fmt_ptr;

    break;
}

}

else {

    *buf_ptr++ = *fmt_ptr;

}

fmt_ptr++;
}

```

```
*buf_ptr = '\0';
```

```
va_end(args);
```

```
HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);
```

```
DWORD bytesWritten;
```

```
WriteConsoleA(hConsole, buffer, (DWORD)(buf_ptr - buffer), &bytesWritten, NULL);
```

```
}
```

```
extern "C" MYLIBRARY_API int power(int base, int exp) {
```

```
    long long result = 1;
```

```
    while (exp > 0) {
```

```
        if (exp % 2 == 1) {
```

```
            result *= base;
```

```
        }
```

```
        base *= base;
```

```
        exp /= 2;
```

```
    }
```

```
    return result;
```

```
}
```

```
extern "C" MYLIBRARY_API void print_blocks(Allocator* a) {
```

```
    my_printf("Allocator size: %d\n", sizeof(Allocator));
```

```
    my_printf("Allocator memory: %d\n", a->memory);
```

```
    for (int i = 0; i < NUM_LISTS; i++) {
```

```
        my_printf("list %d: ", i);
```

```
        if (a->freeLists[i] == NULL) {
```

```
            my_printf("NULL\n");
```

```
        }
```

```
    else {
```

```

    Block* block = a->freeLists[i];

    while (block != NULL) {

        my_printf("block size %d offset %d | ", block->size, (char*)block - (char*)a->memory);

        block = block->next;

    }

}

my_printf("\n");

}

my_printf("\n");

}

```

```

extern "C" MYLIBRARY_API Allocator* allocator_create(void* mem, size_t size) {

```

```

    Allocator* allocator = (Allocator*)mem;

    allocator->total_size = size - sizeof(Allocator);

    allocator->memory = (char*)mem + sizeof(Allocator);

```

```

    for (int i = 0; i < NUM_LISTS; ++i)

        allocator->freeLists[i] = NULL;

```

```

    size_t extent = 0;

```

```

    while (power(2, extent) <= sizeof(Block))

        extent++;

```

```

    allocator->start_extent = extent;

```

```

    size_t offset = 0, pos = 0;

```

```

    size_t p = power(2, extent);

```

```

    size_t blocks_count = 0;

```

```

while (offset + p <= allocator->total_size) {

    Block* block = (Block*)((char*)allocator->memory + offset);

    block->prev = NULL;

    if (allocator->freeLists[pos] == NULL) {

        block->next = NULL;

    }

    else {

        allocator->freeLists[pos]->prev = block;

        block->next = allocator->freeLists[pos];

    }

    allocator->freeLists[pos] = block;

    block->size = p - sizeof(Block);

    my_printf("Block size = %d, offset = %d\n", block->size, offset);

    offset += p;

    blocks_count++;

    if (blocks_count == N_BLOCKS) {

        pos++;

        extent++;

        p = power(2, extent);

        blocks_count = 0;

    }

}

my_printf("\n");

return allocator;

}

extern "C" MYLIBRARY_API void split_block(Allocator* allocator, Block* block) {

    int index = 0;

```

```

while ((1 << index) < block->size + sizeof(Block))

    index++;

my_printf("Block size %d was shared, new size = %d\n", block->size, block->size / 2);

Block* block_copy = (Block*)((char*)block + (block->size + sizeof(Block)) / 2);

if (block->prev != NULL) {
    block->prev->next = block->next;
}
else {
    allocator->freeLists[index] = block->next;
}

if (block->next != NULL)
    block->next->prev = block->prev;

index--;

block->prev = NULL;
block->next = block_copy;
block_copy->prev = block;
block_copy->next = allocator->freeLists[index];

if (allocator->freeLists[index] != NULL) {
    allocator->freeLists[index]->prev = block_copy;
}

allocator->freeLists[index] = block;

block->size = power(2, index) - sizeof(Block);
block_copy->size = power(2, index) - sizeof(Block);

```

```
}
```

```
extern "C" MYLIBRARY_API void* allocator_alloc(Allocator* allocator, size_t size) {  
    int index = 0;  
    while ((1 << (index + allocator->start_extent)) < size + sizeof(Block)) {  
        index++;  
    }  
    my_printf("Allocation block: size %d, index: %d\n", size, index);  
  
    if (index >= NUM_LISTS) {  
        my_printf("Index >= 11\n");  
        return NULL;  
    }  
  
    if (allocator->freeLists[index] != NULL) {  
        Block* block = allocator->freeLists[index];  
        allocator->freeLists[index]->prev = NULL;  
        allocator->freeLists[index] = block->next;  
        my_printf("List index %d has free block size %d offset %d\n", index, block->size, (char*)block -  
(char*)allocator->memory);  
        return block + 1;  
    }  
  
    my_printf("List index %d has not free block, try to find block to split\n", index);  
    int i = index;  
    while (i < NUM_LISTS && allocator->freeLists[i] == NULL)  
        i++;  
  
    if (i == NUM_LISTS) {  
        my_printf("Not found\n");  
    }  
}
```

```

    return NULL;
}

my_printf("Found, index %d\n", i);

for (int j = i; j > index; j--) {
    Block* block = allocator->freeLists[j];
    my_printf("Split block index: %d\n", j);
    split_block(allocator, block);
}
my_printf("return my_malloc\n");
return allocator_alloc(allocator, size);
}

extern "C" MYLIBRARY_API void allocator_free(Allocator* allocator, void* ptr) {

    if (allocator == NULL || ptr == NULL)
        return;

    Block* block = ((Block*)ptr) - 1;

    int index = 0;
    while ((1 << (index + allocator->start_extent)) < block->size + sizeof(Block))
        index++;

    block->prev = NULL;
    if (allocator->freeLists[index] == NULL) {
        block->next = NULL;
    }
    else {

```

```

    allocator->freeLists[index]->prev = block;

    block->next = allocator->freeLists[index];

}

allocator->freeLists[index] = block;

}

extern "C" MYLIBRARY_API void allocator_destroy(Allocator* allocator) {

    if (!allocator)

        return;

}

```

lab44.cpp

```

#include <windows.h>

#include <time.h>

// #include <iostream>

#include "C:/Users/T470/source/repos/DLL_FREEBLOCKS/funcs.h"

typedef int(*AddFunc)(int, int);

typedef Allocator*(*func_create)(void*, const size_t);

typedef void(*func_my_printf)(const char*, ...);

typedef void(*func_destroy)(Allocator*);

typedef void(*func_print)(Block*);

typedef void(*func_print_blocks)(Allocator*);

typedef void*(*func_alloc)(Allocator *, const size_t);

typedef void(*func_free)(Allocator *, void*);

void my_print(const char* format, ...) {

    va_list args;

    va_start(args, format);

    char buffer[1024];

```

```

char* buf_ptr = buffer;

const char* fmt_ptr = format;

int buffer_size = sizeof(buffer);


while (*fmt_ptr) {
    if (*fmt_ptr == '%') {
        fmt_ptr++;
        switch (*fmt_ptr) {
            case 'd': {
                int value = va_arg(args, int);
                char num_buffer[20];
                char* num_ptr = num_buffer;
                if (value < 0) {
                    *buf_ptr++ = '-';
                    value = -value;
                }
                do {
                    *num_ptr++ = (char)((value % 10) + '0');
                    value /= 10;
                } while (value > 0);
                while (num_ptr > num_buffer) {
                    *buf_ptr++ = *--num_ptr;
                }
                break;
            }
            case 'l': {
                unsigned long long value = va_arg(args, unsigned long long);
                char num_buffer[40];
                char* num_ptr = num_buffer;
                do {

```

```

        *num_ptr++ = (char)((value % 10) + '0');

        value /= 10;
    } while (value > 0);
while (num_ptr > num_buffer) {

    *buf_ptr++ = *--num_ptr;

}

break;
}

case 'k': {

    double value = va_arg(args, double);

    char num_buffer[20];

    char* num_ptr = num_buffer;

    if (value < 0) {

        *buf_ptr++ = '-';

        value = -value;

    }


    int afterDot = (value - (int)value) * 100000000;

    int beforeDot = (int)value;


    do {

        *num_ptr++ = (char)((afterDot % 10) + '0');

        afterDot /= 10;

    } while (afterDot > 0);


    *num_ptr++ = '.';


    do {

        *num_ptr++ = (char)((beforeDot % 10) + '0');

        beforeDot /= 10;

```

```
    } while (beforeDot > 0);

    while (num_ptr > num_buffer) {
        *buf_ptr++ = *--num_ptr;
    }

    break;
}

case 's': {
    char* str = va_arg(args, char*);
    while (*str) {
        *buf_ptr++ = *str++;
    }

    break;
}

case 'c': {
    char ch = (char)va_arg(args, int);
    *buf_ptr++ = ch;

    break;
}

case '%': {
    *buf_ptr++ = '%';

    break;
}

default:
    *buf_ptr++ = *fmt_ptr;

    break;
}

else {
    *buf_ptr++ = *fmt_ptr;
```

```

    }

    fmt_ptr++;
}

*buf_ptr = '\0';

va_end(args);

HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);
DWORD bytesWritten;
WriteConsoleA(hConsole, buffer, (DWORD)(buf_ptr - buffer), &bytesWritten, NULL);
}

```

```

LPWSTR ConvertToWideString(const char* str) {
    if (str == nullptr) return nullptr;

    int size_needed = MultiByteToWideChar(CP_UTF8, 0, str, -1, NULL, 0);
    LPWSTR wideString = new wchar_t[size_needed];
    MultiByteToWideChar(CP_UTF8, 0, str, -1, wideString, size_needed);
    return wideString;
}

```

```

Allocator* system_allocator_create(void* memory, size_t size) {
    my_print("System allocator was created\n");
    return (Allocator*)memory;
}

```

```

void system_allocator_destroy(Allocator* allocator) {
    my_print("System allocator was destroyed\n");
    // Ничего не делаем для системного аллокатора
}

```

```
void* system_allocator_alloc(Allocator* allocator, size_t size) {  
    my_print("System alloc %d bytes\n", size);  
    return VirtualAlloc(NULL, size, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);  
}
```

```
void system_allocator_free(Allocator* allocator, void* memory) {  
    my_print("System free\n");  
    VirtualFree(memory, 0, MEM_RELEASE);  
}
```

```
void system_print_blocks(Allocator* A) {  
    my_print("System print blocks\n");  
}
```

//path 1: C:/Users/T470/source/repos/DLL_FREEBLOCKS/x64/Debug/DLL_FREEBLOCKS.dll

//path 2: C:/Users/T470/source/repos/Dll_POW2/x64/Debug/Dll_POW2.dll

```
int main(int argc, char* argv[]) {  
    if (argc > 2) {  
        my_print("Incorrect number of arguments");  
        return 3;  
    }  
    if (argc == 2) {  
        HMODULE hModule = LoadLibrary(ConvertToWideString(argv[1]));  
        if (hModule) {  
            func_print_blocks p_blocks = (func_print_blocks)GetProcAddress(hModule, "print_blocks");  
            if (!p_blocks) {  
                my_print("Could not locate the print_blocks function.");  
                FreeLibrary(hModule);  
                return 1;  
            }  
        }  
    }  
}
```

```

}

func_create create = (func_create)GetProcAddress(hModule, "allocator_create");

if (!create) {
    my_print("Could not locate the create function.");
    FreeLibrary(hModule);
    return 1;
}

func_alloc alloc = (func_alloc)GetProcAddress(hModule, "allocator_alloc");

if (!alloc) {
    my_print("Could not locate the alloc function.");
    FreeLibrary(hModule);
    return 1;
}

func_free free_func = (func_free)GetProcAddress(hModule, "allocator_free");

if (!free_func) {
    my_print("Could not locate the free function.");
    FreeLibrary(hModule);
    return 1;
}

func_destroy destroy = (func_destroy)GetProcAddress(hModule, "allocator_destroy");

if (!destroy) {
    my_print("Could not locate the destroy function.");
    FreeLibrary(hModule);
    return 1;
}

}

//char data[MAX_SIZE];

void* data = VirtualAlloc(NULL, MAX_SIZE, MEM_COMMIT | MEM_RESERVE,
PAGE_READWRITE);

if (!data) {

```

```

    my_print("Virtual alloc error");

    FreeLibrary(hModule);

    return 2;
}

Allocator* a = create(data, MAX_SIZE);

p_blocks(a);


clock_t start = clock();

char* pc[10];

for (int i = 0; i < 10; i++) {
    pc[i] = (char*)alloc(a, 10);

    if (!pc[i]) {
        my_print("Allocation error");
    }

    p_blocks(a);
}


clock_t end = clock();

my_print("Allocation time: %k seconds\n", (double)(end - start) / CLOCKS_PER_SEC);

start = clock();


for (int i = 0; i < 10; i++) {
    free_func(a, (void*)pc[i]);

    p_blocks(a);
}


end = clock();

my_print("Free time: %k seconds\n", (double)(end - start) / CLOCKS_PER_SEC);


destroy(a);

```

```

    VirtualFree(data, 0, MEM_RELEASE);

    FreeLibrary(hModule);
}
}

else {

    my_print("Could not load the DLL, using system funcs.\n\n");

    func_print_blocks p_blocks = (func_print_blocks)system_print_blocks;

    func_create create = (func_create)system_allocator_create;

    func_alloc alloc = (func_alloc)system_allocator_alloc;

    func_free free_func = (func_free)system_allocator_free;

    func_destroy destroy = (func_destroy)system_allocator_destroy;


    void* data = VirtualAlloc(NULL, MAX_SIZE, MEM_COMMIT | MEM_RESERVE,
PAGE_READWRITE);

    if (!data) {

        my_print("Virtual alloc error");

        return 2;

    }


    Allocator* a = create(data, MAX_SIZE);

    p_blocks(a);


    clock_t start = clock();

    char* pc[10];

    for (int i = 0; i < 10; i++) {

        pc[i] = (char*)alloc(a, 10);

        if (!pc[i]) {

            my_print("Allocation error");

        }

        p_blocks(a);

```

```

}

clock_t end = clock();

my_print("Allocation time: %k seconds\n", (double)(end - start) / CLOCKS_PER_SEC);

start = clock();

for (int i = 0; i < 10; i++) {
    free_func(a, (void*)pc[i]);
    p_blocks(a);
}

end = clock();

my_print("Free time: %k seconds\n", (double)(end - start) / CLOCKS_PER_SEC);

destroy(a);

VirtualFree(data, 0, MEM_RELEASE);

}

return 0;

}

```

Протокол работы программы

NtTrace:

Process 5596 starting at 00007FF6A5A012DA with command line: "lab44.exe"

C:\Users\T470\source\repos\lab44\x64\Debug\lab44.exe

Loaded DLL at 00007FF8E5470000 C:\Windows\SYSTEM32\ntdll.dll

NtQueryPerformanceCounter(Counter=0x901237f7d0 [8.72141e+12], Freq=null) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f818 [0x00007ff8e55f4000], Size=0x901237f810 [0x1000], NewProtect=4, OldProtect=0x901237f850 [8]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f818 [0x00007ff8e55f4000], Size=0x901237f810 [0x1000], NewProtect=8, OldProtect=0x901237f850 [4]) => 0

```

NtCreateEvent(EventHandle=0x7ff8e55dc478 [8],
DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|SYNCHRONIZE|0x3,
ObjectAttributes=null, EventType=0 [NotificationEvent], InitialState=false) => 0

NtQuerySystemInformation(SystemInformationClass=0 [SystemBasicInformation],
SystemInformation=0x901237f780, Length=0x40, ReturnLength=null) => 0

NtQueryInformationProcess(ProcessHandle=-1, ProcessInformationClass=0x24 [ProcessCookie],
ProcessInformation=0x901237f4a8, Length=4, ReturnLength=null) => 0

NtQuerySystemInformation(SystemInformationClass=0xc0 [SystemFlushInformation],
SystemInformation=0x901237f380, Length=0x20, ReturnLength=null) => 0

NtQueryVirtualMemory(ProcessHandle=-1, BaseAddress=0x7ff8e5470000, MemoryInformationClass=6
[MemoryImageInformation], MemoryInformation=0x901237f290, Length=0x18, ReturnLength=null) =>
0

NtQueryVirtualMemory(ProcessHandle=-1, BaseAddress=null, MemoryInformationClass=4
[MemoryWorkingSetExInformation], MemoryInformation=0x901237f350, Length=0x50,
ReturnLength=null) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f3a0 [0x00007ff8e55f1000],
Size=0x901237f398 [0x4000], NewProtect=2, OldProtect=0x901237f390 [4]) => 0

NtOpenKey(KeyHandle=0x901237dfe0 [0xc], DesiredAccess=0x1,
ObjectAttributes="\Registry\MACHINE\System\CurrentControlSet\Control\Session Manager") => 0

NtQueryValueKey(KeyHandle=0xc, ValueName="RaiseExceptionOnPossibleDeadlock",
KeyValueInformationClass=2 [KeyValuePartialInformation], KeyValueInformation=0x901237dff0,
Length=0x50, ResultLength=0x901237dfe8) => 0xc0000034 [2 '═х єфрхЄё эрщЄш єърчрээ√щ Ърщы.']

NtClose(Handle=0xc) => 0

NtOpenKey(KeyHandle=0x901237df78 [0x10], DesiredAccess=0x9,
ObjectAttributes="\Registry\Machine\Software\Microsoft\Windows NT\CurrentVersion\Image File
Execution Options") => 0

NtOpenKey(KeyHandle=0x901237e060, DesiredAccess=0x9, ObjectAttributes=0x10:"lab44.exe") =>
0xc0000034 [2 '═х єфрхЄё эрщЄш єърчрээ√щ Ърщы.']

NtOpenKey(KeyHandle=0x901237dfc0, DesiredAccess=0x1,
ObjectAttributes="\Registry\Machine\SYSTEM\CurrentControlSet\Control\Session Manager\Segment
Heap") => 0xc0000034 [2 '═х єфрхЄё эрщЄш єърчрээ√щ Ърщы.']

NtQueryInformationProcess(ProcessHandle=-1, ProcessInformationClass=0x24 [ProcessCookie],
ProcessInformation=0x7ff8e55dd230, Length=4, ReturnLength=null) => 0

NtQueryInformationProcess(ProcessHandle=-1, ProcessInformationClass=0x24 [ProcessCookie],
ProcessInformation=0x901237f338, Length=4, ReturnLength=null) => 0

NtQuerySecurityAttributesToken(TokenHandle=-4, Attributes="WIN://SYSAPPID",
NumberOfAttributes=2, Buffer=0x901237ed70, Length=0x330, ReturnLength=0x901237ed28) =>
0xc0000225 [1168 ' ЫХЪХЭЄ эх эрщфхэ.']

```

NtQuerySecurityAttributesToken(TokenHandle=-4, Attributes="WIN://SYSAPPID",
NumberOfAttributes=1, Buffer=0x901237ed70, Length=0x330, ReturnLength=0x901237ed28) =>
0xc0000225 [1168 ыхьхэЄ эх эрщфхэ.]

NtOpenKey(KeyHandle=0x901237f2b0 [0xc], DesiredAccess=0x9,
ObjectAttributes="\Registry\Machine\SYSTEM\CurrentControlSet\Control\Session Manager") => 0

NtQueryValueKey(KeyHandle=0xc, ValueName="ResourcePolicies", KeyValueInformationClass=2
[KeyValuePartialInformation], KeyValueInformation=0x901237f2f0, Length=0x18,
ResultLength=0x901237f2b8) => 0xc0000034 [2 'x ефрхЄё эрщЄш єърчрээ\щ їрщы.]

NtClose(Handle=0xc) => 0

NtQueryInformationProcess(ProcessHandle=-1, ProcessInformationClass=0x24 [ProcessCookie],
ProcessInformation=0x901237f390, Length=4, ReturnLength=null) => 0

NtQuerySystemInformation(SystemInformationClass=0 [SystemBasicInformation],
SystemInformation=0x901237f330, Length=0x40, ReturnLength=null) => 0

NtQuerySystemInformation(SystemInformationClass=0x3e [SystemEmulationBasicInformation],
SystemInformation=0x901237f360, Length=0x40, ReturnLength=null) => 0

NtAllocateVirtualMemoryEx(ProcessHandle=-1, lpAddress=0x7ff8e55ddc38 [0x00007ff5d64a0000],
ZeroBits=0x000000901237f2e0, pSize=0x102000 [0], flAllocationType=4, DataBuffer=0x901237f248,
DataCount=1) => 0

NtAllocateVirtualMemoryEx(ProcessHandle=-1, lpAddress=0x7ff8e55ddc30 [0x00007ff5d84a0000],
ZeroBits=0x000000901237f2e8, pSize=0x1000 [0], flAllocationType=4, DataBuffer=null, DataCount=0)
=> 0

NtAllocateVirtualMemoryEx(ProcessHandle=-1, lpAddress=0x7ff8e55ddca0 [0x00007ff4d6480000],
ZeroBits=0x000000901237f290, pSize=0x102000 [0], flAllocationType=4, DataBuffer=0x901237f1f8,
DataCount=1) => 0

NtQuerySystemInformation(SystemInformationClass=0 [SystemBasicInformation],
SystemInformation=0x901237f1d0, Length=0x40, ReturnLength=null) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237ed90 [0x0000026059650000],
ZeroBits=0, pSize=0x901237ed98 [0x00170000], flAllocationType=0x2000, flProtect=4) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237ed90 [0x0000026059650000],
pSize=0x901237ed88 [0x00070000], flFreeType=0x8000) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237ed78 [0x00000260596c0000],
ZeroBits=0, pSize=0x901237ed70 [0x2000], flAllocationType=0x1000, flProtect=4) => 0

NtQuerySystemInformation(SystemInformationClass=0xc5 [SystemHypervisorSharedPageInformation],
SystemInformation=0x901237f538, Length=8, ReturnLength=null) => 0

NtQuerySystemInformation(SystemInformationClass=0x37 [SystemNumaProcessorMap],
SystemInformation=0x901237efa0, Length=0x408, ReturnLength=0x901237f3c0 [0x18]) => 0

NtCreateEvent(EventHandle=0x901237f1d8 [0xc],
DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|SYNCHRONIZE|0x3,
ObjectAttributes=null, EventType=1 [SynchronizationEvent], InitialState=false) => 0

NtCreateWaitCompletionPacket(WaitCompletionPacketHandle=0x260596c0b20 [0x14],
DesiredAccess=0x1, ObjectAttributes=null) => 0

NtQuerySystemInformationEx(SystemInformationClass=0x6b
[SystemLogicalProcessorAndGroupInformation], QueryType=0x901237ef30 [1], Alignment=4,
SystemInformation=0x260596c0ed0, Length=0x50, ReturnLength=0x901237ef28 [0x30]) => 0

NtCreateIoCompletion(IoHandle=0x260596c0c00 [0x18],
DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|SYNCHRONIZE|0x3,
ObjectAttributes=null, NumberOfConcurrentThreads=5) => 0

NtCreateWorkerFactory(WorkerFactoryHandleReturn=0x260596c0bf8 [0x1c],
DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|0xff,
ObjectAttributes=null, CompletionPortHandle=0x18, WorkerProcessHandle=-1,
StartRoutine=0x7ff8e54bd110, StartParameter=0x260596c0bc0, MaxThreadCount=0x200,
StackReserve=0x00100000, StackCommit=0x1000) => 0

NtCreateTimer2(TimerHandle=0x260596c0c50 [0x20], Unknown1=null, ObjectAttributes=null,
Attributes=8, DesiredAccess=SYNCHRONIZE|0x7ff800000002) => 0

NtCreateWaitCompletionPacket(WaitCompletionPacketHandle=0x260596c0c58 [0x24],
DesiredAccess=0x1, ObjectAttributes=null) => 0

NtAssociateWaitCompletionPacket(WaitCompletionPacketHandle=0x24, IoCompletionHandle=0x18,
TargetObjectHandle=0x20, KeyContext=0x260596c0c60, ApcContext=0x260596c0c30, IoStatus=0,
IoStatusInformation=1, AlreadySignaled=0x901237eef0 [0]) => 0

NtCreateTimer2(TimerHandle=0x260596c0cc8 [0x28], Unknown1=null, ObjectAttributes=null,
Attributes=8, DesiredAccess=SYNCHRONIZE|0x260000000002) => 0

NtCreateWaitCompletionPacket(WaitCompletionPacketHandle=0x260596c0cd0 [0x2c],
DesiredAccess=0x1, ObjectAttributes=null) => 0

NtAssociateWaitCompletionPacket(WaitCompletionPacketHandle=0x2c, IoCompletionHandle=0x18,
TargetObjectHandle=0x28, KeyContext=0x260596c0cd8, ApcContext=0x260596c0c30, IoStatus=0,
IoStatusInformation=0, AlreadySignaled=0x901237eef0 [0]) => 0

NtSetInformationWorkerFactory(WorkerFactoryHandle=0x1c, InformationClass=5
[WorkerFactoryThreadMaximum], Buffer=0x901237eff8, BufferLength=4) => 0

NtSetInformationWorkerFactory(WorkerFactoryHandle=0x1c, InformationClass=0xe
[WorkerFactoryThreadSoftMaximum], Buffer=0x901237eff8, BufferLength=4) => 0

NtSetInformationWorkerFactory(WorkerFactoryHandle=0x1c, InformationClass=3
[WorkerFactoryBindingCount], Buffer=0x901237f118, BufferLength=4) => 0

NtAssociateWaitCompletionPacket(WaitCompletionPacketHandle=0x14, IoCompletionHandle=0x18, TargetObjectHandle=0xc, KeyContext=0x260596c0b38, ApcContext=0x260596c09b0, IoStatus=0x0000026000000000, IoStatusInformation=0, AlreadySignaled=0x901237f160 [0x596c0b00]) => 0

NtTraceControl(CtrlCode=0x1b, InputBuffer=0x901237f218, InputBufferLength=4, OutputBuffer=null, OutputBufferLength=0, ReturnLength=0x901237f1d0 [0]) => 0

NtTraceControl(CtrlCode=0xf, InputBuffer=0x901237f278, InputBufferLength=0xa0, OutputBuffer=0x901237f278, OutputBufferLength=0xa0, ReturnLength=0x901237f270 [0xa0]) => 0

NtTraceControl(CtrlCode=0xf, InputBuffer=0x901237f278, InputBufferLength=0xa0, OutputBuffer=0x901237f278, OutputBufferLength=0xa0, ReturnLength=0x901237f270 [0xa0]) => 0

NtTraceControl(CtrlCode=0xf, InputBuffer=0x901237f278, InputBufferLength=0xa0, OutputBuffer=0x901237f278, OutputBufferLength=0xa0, ReturnLength=0x901237f270 [0xa0]) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237ebe0 [0x00000260596c2000], ZeroBits=0, pSize=0x901237ec88 [0x2000], flAllocationType=0x1000, flProtect=4) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f3c0 [0x00000260595e0000], pSize=0x901237f3c8 [0x00020000], flFreeType=0x8000) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f3a0 [0x00007ff8e55f1000], Size=0x901237f398 [0x4000], NewProtect=4, OldProtect=0x901237f390 [2]) => 0

NtOpenDirectoryObject(DirectoryHandle=0x7ff8e55f1298 [0x3c], DesiredAccess=0x3, ObjectAttributes="\KnownDlls") => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f3a0 [0x00007ff8e55f1000], Size=0x901237f398 [0x4000], NewProtect=2, OldProtect=0x901237f390 [4]) => 0

NtOpenSymbolicLinkObject(LinkHandle=0x901237f4f8 [0x40], DesiredAccess=0x1, ObjectAttributes=0x3c:"KnownDllPath") => 0

NtQuerySymbolicLinkObject(LinkHandle=0x40, LinkTarget="C:\\Windows\\System32", ReturnedLength=0x901237f4ac [0x28]) => 0

NtClose(Handle=0x40) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f370 [0x00007ff8e55f1000], Size=0x901237f368 [0x4000], NewProtect=4, OldProtect=0x901237f360 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f3a0 [0x00007ff8e55f1000], Size=0x901237f398 [0x4000], NewProtect=2, OldProtect=0x901237f390 [4]) => 0

NtCreateEvent(EventHandle=0x7ff8e55dc380 [0x44], DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|SYNCHRONIZE|0x3, ObjectAttributes=null, EventType=1 [SynchronizationEvent], InitialState=false) => 0

NtCreateEvent(EventHandle=0x7ff8e55dc3b0 [0x48], DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|SYNCHRONIZE|0x3, ObjectAttributes=null, EventType=1 [SynchronizationEvent], InitialState=false) => 0

NtQueryPerformanceCounter(Counter=0x901237f2f0 [8.72141e+12], Freq=null) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f300 [0x00007ff8e55f1000], Size=0x901237f2f8 [0x4000], NewProtect=4, OldProtect=0x901237f2f0 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f300 [0x00007ff8e55f1000], Size=0x901237f2f8 [0x4000], NewProtect=2, OldProtect=0x901237f2f0 [4]) => 0

NtOpenFile(FileHandle=0x901237f3a8 [0x4c], DesiredAccess=SYNCHRONIZE[0x20, ObjectAttributes="\??\C:\Users\T470\source\repos\lab44\x64\Debug", IoStatusBlock=0x901237f318 [0/1], ShareAccess=3, OpenOptions=0x21) => 0

NtQueryVolumeInformationFile(FileHandle=0x4c, IoStatusBlock=0x901237f318 [0/8], FsInformation=0x901237f300, Length=8, FsInformationClass=4 [FsDeviceInformation]) => 0

NtSetEvent(EventHandle=0x44, PrevState=null) => 0

NtSetInformationProcess(ProcessHandle=-1, ProcessInformationClass=0x5d [ProcessLoaderDetour], ProcessInformation=0x901237f060, Length=4) => 0

NtOpenSection(SectionHandle=0x901237f038 [0x50], DesiredAccess=0xd, ObjectAttributes=0x3c:"KERNEL32.DLL") => 0

Loaded DLL at 00007FF8E36A0000 C:\Windows\System32\KERNEL32.DLL

NtMapViewOfSection(SectionHandle=0x50, ProcessHandle=-1, BaseAddress=0x260596c3280 [0x00007ff8e36a0000], ZeroBits=0, CommitSize=0, SectionOffset=null, ViewSize=0x260596c31e0 [0x000c2000], InheritDisposition=1 [ViewShare], AllocationType=0x00800000, Protect=0x80) => 0

NtQueryPerformanceCounter(Counter=0x901237eeb0 [8.72141e+12], Freq=null) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237eec8 [0x00007ff8e375f000], Size=0x901237eec0 [0x1000], NewProtect=2, OldProtect=0x901237ef30 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237eec0 [0x00007ff8e55f1000], Size=0x901237eeb8 [0x4000], NewProtect=4, OldProtect=0x901237eeb0 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237eec0 [0x00007ff8e55f1000], Size=0x901237eeb8 [0x4000], NewProtect=2, OldProtect=0x901237eeb0 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ef10 [0x00007ff8e3723000], Size=0x901237ef18 [0x4000], NewProtect=4, OldProtect=0x260596c31c8 [2]) => 0

NtOpenSection(SectionHandle=0x901237e9a8 [0x60], DesiredAccess=0xd, ObjectAttributes=0x3c:"KERNELBASE.dll") => 0

Loaded DLL at 00007FF8E2F90000 C:\Windows\System32\KERNELBASE.dll

NtMapViewOfSection(SectionHandle=0x60, ProcessHandle=-1, BaseAddress=0x260596c3960 [0x00007ff8e2f90000], ZeroBits=0, CommitSize=0, SectionOffset=null, ViewSize=0x260596c38c0 [0x002fe000], InheritDisposition=1 [ViewShare], AllocationType=0x00800000, Protect=0x80) => 0

NtQueryPerformanceCounter(Counter=0x901237e820 [8.72141e+12], Freq=null) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237e838 [0x00007ff8e3263000], Size=0x901237e830 [0x1000], NewProtect=2, OldProtect=0x901237e8a0 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237e830 [0x00007ff8e55f1000], Size=0x901237e828 [0x4000], NewProtect=4, OldProtect=0x901237e820 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237e830 [0x00007ff8e55f1000], Size=0x901237e828 [0x4000], NewProtect=2, OldProtect=0x901237e820 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237e880 [0x00007ff8e317a000], Size=0x901237e888 [0x3000], NewProtect=4, OldProtect=0x260596c38a8 [2]) => 0

NtClose(Handle=0x60) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x260596c31a8 [0x00007ff8e3723000], Size=0x260596c31b0 [0x4000], NewProtect=2, OldProtect=0x901237ee00 [4]) => 0

NtClose(Handle=0x50) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x260596c3888 [0x00007ff8e317a000], Size=0x260596c3890 [0x3000], NewProtect=2, OldProtect=0x901237ef00 [4]) => 0

NtSetInformationProcess(ProcessHandle=-1, ProcessInformationClass=0x23 [ProcessTlsInformation], ProcessInformation=0x901237ee60, Length=0x28) => 0

NtApphelpCacheControl(ServiceClass=6, ServiceData="") => 0

NtApphelpCacheControl(ServiceClass=6, ServiceData="") => 0

NtSetInformationVirtualMemory(ProcessHandle=-1, VmInformationClass=4, NumberOfEntries=1, VirtualAddresses=0x901237ef20, VmInformation=0x901237eff8, VmInformationLength=4) => 0xc00000bb [50 'Прыющ чряЁюё эх яюфхЁцштрхЄё .']

NtQuerySystemInformation(SystemInformationClass=0x32 [SystemRangeStartInformation], SystemInformation=0x901237eb80, Length=8, ReturnLength=null) => 0

NtQuerySystemInformation(SystemInformationClass=0 [SystemBasicInformation], SystemInformation=0x7ff8e324ee60, Length=0x40, ReturnLength=null) => 0

NtOpenSection(SectionHandle=0x901237e940 [0x40], DesiredAccess=0x4, ObjectAttributes="\Sessions\6\Windows\SharedSection") => 0

NtCreateSection(SectionHandle=0x901237e960 [0x50], DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|0x1f, ObjectAttributes=null, SectionSize=0x901237e950 [65536], Protect=4, Attributes=0x08000000, FileHandle=0) => 0

NtConnectPort(PortHandle=0x7ff8e55dcc48 [0x60], PortName="\Sessions\6\Windows\ApiPort", SecurityQos=0x901237ea80, ClientView=0x901237e978, ServerView=0x901237e9a8, MaxMsgLength=0x901237e970 [0x3b8], ConnectionInfo=0x901237e9f0, ConnectionInfoLength=0x901237e948 [0x30]) => 0

NtClose(Handle=0x50) => 0

NtMapViewOfSection(SectionHandle=0x40, ProcessHandle=-1, BaseAddress=0x901237e958 [0x00007ff4d6380000], ZeroBits=0, CommitSize=0, SectionOffset=null, ViewSize=0x901237e968 [0x00100000], InheritDisposition=2 [ViewUnmap], AllocationType=0x00500000, Protect=2) => 0

NtClose(Handle=0x40) => 0

NtQueryVirtualMemory(ProcessHandle=-1, BaseAddress=0x260595e0000, MemoryInformationClass=0 [MemoryBasicInformation], MemoryInformation=0x901237e630, Length=0x30, ReturnLength=null) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, IpAddress=0x901237e340 [0x00000260596c4000], ZeroBits=0, pSize=0x901237e3e8 [0x1000], flAllocationType=0x1000, flProtect=4) => 0

NtInitializeNlsFiles(BaseAddress=0x901237eb20 [0x00000260597c0000], DefaultLocaleId=0x7ff8e3250900 [0x419], DefaultCasingTableSize=null) => 0

NtCreateFile(FileHandle=0x901237eb88 [0x40], DesiredAccess=READ_CONTROL|SYNCHRONIZE|0x19f, ObjectAttributes=4:"\Connect", IoStatusBlock=0x901237e540 [0/0x18], AllocationSize=null, FileAttributes=0, ShareAccess=7, CreateDisposition=2, CreateOptions=0x20, EaBuffer=0x260596c47e0, EaLength=0x54b) => 0

NtDeviceIoControlFile(FileHandle=0x40, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237ead0 [0/0], IoControlCode=0x00500023, InputBuffer=null, InputBufferLength=0, OutputBuffer=0x901237eaf0, OutputBufferLength=8) => 0

NtSetInformationProcess(ProcessHandle=-1, ProcessInformationClass=0x31 [ProcessOwnerInformation], ProcessInformation=0x901237eaf8, Length=8) => 0

NtDeviceIoControlFile(FileHandle=0x40, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e8c0, IoControlCode=0x00500016, InputBuffer=0x901237e8d0, InputBufferLength=0x30, OutputBuffer=null, OutputBufferLength=0) => 0xc00700bb [187 '==x эрщфхэю єърчрэюх шь ёшёЄхьэюю ёхьрІюЁр.']

NtTraceControl(CtrlCode=0xf, InputBuffer=0x901237e9e8, InputBufferLength=0xa0, OutputBuffer=0x901237e9e8, OutputBufferLength=0xa0, ReturnLength=0x901237e9e0 [0xa0]) => 0

NtTraceControl(CtrlCode=0x1e, InputBuffer=0x901237ea28, InputBufferLength=0x18, OutputBuffer=0x901237ea40, OutputBufferLength=0x78, ReturnLength=0x901237ea20 [0]) => 0

NtOpenProcessToken(ProcessHandle=-1, DesiredAccess=0x8, TokenHandle=0x901237ea20 [0x64]) => 0

NtQueryInformationToken(TokenHandle=0x64, TokenInformationClass=0xc [TokenSessionId], TokenInformation=0x901237e340, Length=4, ReturnLength=0x901237e320 [4]) => 0

NtQueryInformationToken(TokenHandle=0x64, TokenInformationClass=0x1d [TokenIsAppContainer], TokenInformation=0x901237e388, Length=4, ReturnLength=0x901237e320 [4]) => 0

NtQueryInformationToken(TokenHandle=0x64, TokenInformationClass=0x2a [TokenPrivateNameSpace], TokenInformation=0x901237e324, Length=4, ReturnLength=0x901237e320 [4]) => 0

NtOpenDirectoryObject(DirectoryHandle=0x901237e348 [0x68], DesiredAccess=0xf, ObjectAttributes="\Sessions\6\BaseNamedObjects") => 0

NtQueryInformationToken(TokenHandle=0x64, TokenInformationClass=0x2c [TokenBnoIsolation], TokenInformation=0x901237e640, Length=0x120, ReturnLength=0x901237e320 [0x10]) => 0

NtClose(Handle=0x64) => 0

NtCreateMutant(MutantHandle=0x901237ea78 [0x6c], DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|SYNCHRONIZE|0x1, ObjectAttributes=0x68:"Local\SM0:5596:304:WilStaging_02", InitialOwner=false) => 0

NtWaitForSingleObject(Handle=0x6c, Alertable=false, Timeout=null) => 0

NtOpenSemaphore(SemaphoreHandle=0x901237e868, DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|SYNCHRONIZE|0x3, ObjectAttributes=0x68:"Local\SM0:5596:304:WilStaging_02_p0") => 0xc0000034 [2 'x
€фpx€ё эрщЄш єърчрээ√щ їрщы.']

NtCreateSemaphore(SemaphoreHandle=0x901237e758 [0x64], DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|SYNCHRONIZE|0x3, ObjectAttributes=0x68:"Local\SM0:5596:304:WilStaging_02_p0", InitialCount=0x165b1244, MaxCount=0x165b1244) => 0

NtCreateSemaphore(SemaphoreHandle=0x901237e758 [0x70], DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|SYNCHRONIZE|0x3, ObjectAttributes=0x68:"Local\SM0:5596:304:WilStaging_02_p0h", InitialCount=0x130, MaxCount=0x130) => 0

NtReleaseMutant(MutantHandle=0x6c, PreviousCount=null) => 0

NtQueryWnfStateData(StateName=0x901237ead0 [0xa3bc0875], TypeId=0x901237eb78, ExplicitScope=null, ChangeStamp=0x901237eac4 [8], Buffer=0x901237dac0, BufferSize=0x901237eac0 [8]) => 0

NtCreateEvent(EventHandle=0x901237ea30 [0x74], DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|SYNCHRONIZE|0x3, ObjectAttributes=null, EventType=1 [SynchronizationEvent], InitialState=false) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237e1f0 [0x00000260596c5000], ZeroBits=0, pSize=0x901237e298 [0x1000], flAllocationType=0x1000, flProtect=4) => 0

NtCreateWaitCompletionPacket(WaitCompletionPacketHandle=0x260596c4fb0 [0x78], DesiredAccess=0x1, ObjectAttributes=null) => 0

NtSetWnfProcessNotificationEvent(NotificationEvent=0x74) => 0

NtAssociateWaitCompletionPacket(WaitCompletionPacketHandle=0x78, IoCompletionHandle=0x18, TargetObjectHandle=0x74, KeyContext=0x260596c4fc8, ApcContext=0x260596c4e40, IoStatus=0x0000026000000000, IoStatusInformation=0, AlreadySignaled=0x901237e9b0 [0x596c0b00]) => 0

NtSubscribeWnfStateChange(StateName=0x260596c5140 [0xa3bc0875], ChangeStamp=8, EventMask=0x11, SubscriptionId=0x901237eaa0 [0x000218e0]) => 0

NtQueryWnfStateData(StateName=0x901237ec10 [0xa3bc7c75], TypeId=null, ExplicitScope=null, ChangeStamp=0x901237ec08 [0], Buffer=null, BufferSize=0x901237ec0c [0]) => 0

NtSubscribeWnfStateChange(StateName=0x260596c5470 [0xa3bc7c75], ChangeStamp=0, EventMask=0x11, SubscriptionId=0x901237ea80 [0x000218e1]) => 0

NtQueryWnfStateData(StateName=0x901237ec10 [0xa3bc88f5], TypeId=null, ExplicitScope=null, ChangeStamp=0x901237ec08 [0], Buffer=null, BufferSize=0x901237ec0c [0]) => 0

NtSubscribeWnfStateChange(StateName=0x260596c5620 [0xa3bc88f5], ChangeStamp=0, EventMask=0x11, SubscriptionId=0x901237ea80 [0x000218e2]) => 0

NtQuerySystemInformationEx(SystemInformationClass=0xd3 [SystemFeatureConfigurationSectionInformation], QueryType=0x901237e9c0 [0], Alignment=0x18, SystemInformation=0x901237e9e0, Length=0x50, ReturnLength=null) => 0

NtMapViewOfSection(SectionHandle=0x7c, ProcessHandle=-1, BaseAddress=0x901237e990 [0x00000260595f0000], ZeroBits=0, CommitSize=0, SectionOffset=null, ViewSize=0x901237e998 [0x1000], InheritDisposition=2 [ViewUnmap], AllocationType=0, Protect=2) => 0

NtMapViewOfSection(SectionHandle=0x80, ProcessHandle=-1, BaseAddress=0x901237e990 [0x0000026059650000], ZeroBits=0, CommitSize=0, SectionOffset=null, ViewSize=0x901237e998 [0x1000], InheritDisposition=2 [ViewUnmap], AllocationType=0, Protect=2) => 0

NtMapViewOfSection(SectionHandle=0x84, ProcessHandle=-1, BaseAddress=0x901237e990 [0x0000026059660000], ZeroBits=0, CommitSize=0, SectionOffset=null, ViewSize=0x901237e998 [0x1000], InheritDisposition=2 [ViewUnmap], AllocationType=0, Protect=2) => 0

NtClose(Handle=0x7c) => 0

NtClose(Handle=0x80) => 0

NtClose(Handle=0x84) => 0

NtQueryWnfStateData(StateName=0x901237e9c8 [0xa3bc7c75], TypeId=null, ExplicitScope=null, ChangeStamp=0x901237ea58 [0], Buffer=0x901237eab0, BufferSize=0x901237e9b0 [0]) => 0

NtSetTimer2(TimerHandle=0x28, DueTime=0x901237ea40 [-3e+09], Period=null, Parameters=0x901237ea48) => 0

NtOpenKey(KeyHandle=0x901237ec10, DesiredAccess=KEY_READ, ObjectAttributes="\Registry\Machine\System\CurrentControlSet\Control\StateSeparation\RedirectonMap\Keys") => 0xc0000034 [2 'x ефрхЄё эрщЄш єърчрээ\щ їрщы.']

NtTraceControl(CtrlCode=0xf, InputBuffer=0x901237ec28, InputBufferLength=0xa0, OutputBuffer=0x901237ec28, OutputBufferLength=0xa0, ReturnLength=0x901237ec20 [0xa0]) => 0

NtTraceControl(CtrlCode=0x1e, InputBuffer=0x901237ec68, InputBufferLength=0x18, OutputBuffer=0x901237ec80, OutputBufferLength=0x78, ReturnLength=0x901237ec60 [0]) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237e540 [0x00000260596c6000], ZeroBits=0, pSize=0x901237e5e8 [0x1000], flAllocationType=0x1000, flProtect=4) => 0

NtTraceControl(CtrlCode=0xf, InputBuffer=0x901237ec58, InputBufferLength=0xa0, OutputBuffer=0x901237ec58, OutputBufferLength=0xa0, ReturnLength=0x901237ec50 [0xa0]) => 0

NtTraceControl(CtrlCode=0x1e, InputBuffer=0x901237ec98, InputBufferLength=0x18, OutputBuffer=0x901237ecb0, OutputBufferLength=0x78, ReturnLength=0x901237ec90 [0]) => 0

NtSetInformationVirtualMemory(ProcessHandle=-1, VmInformationClass=4, NumberOfEntries=1, VirtualAddresses=0x901237ef60, VmInformation=0x901237f038, VmInformationLength=4) => 0xc00000bb [50 'Прыюц чряЁюё эх яюфхЁцштрхЄё .']

NtOpenKey(KeyHandle=0x901237eb40, DesiredAccess=0x9, ObjectAttributes=0x10:"lab44.exe") => 0xc0000034 [2 '═х єфрхЄё эрщЄш єърчрээ√щ Ырщы.']

NtTraceControl(CtrlCode=0xf, InputBuffer=0x901237ea48, InputBufferLength=0xa0, OutputBuffer=0x901237ea48, OutputBufferLength=0xa0, ReturnLength=0x901237ea40 [0xa0]) => 0

NtTraceControl(CtrlCode=0x1e, InputBuffer=0x901237ea88, InputBufferLength=0x18, OutputBuffer=0x901237eaa0, OutputBufferLength=0x78, ReturnLength=0x901237ea80 [0]) => 0

NtQuerySystemInformation(SystemInformationClass=0 [SystemBasicInformation], SystemInformation=0x7ff8e3757aa0, Length=0x40, ReturnLength=null) => 0

NtSetEvent(EventHandle=0x44, PrevState=null) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=4, OldProtect=0x901237ec88 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=2, OldProtect=0x901237ec88 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=4, OldProtect=0x901237ec88 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=2, OldProtect=0x901237ec88 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=4, OldProtect=0x901237ec88 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=2, OldProtect=0x901237ec88 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=4, OldProtect=0x901237ec88 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=2, OldProtect=0x901237ec88 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=4, OldProtect=0x901237ec88 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=2, OldProtect=0x901237ec88 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=4, OldProtect=0x901237ec88 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=2, OldProtect=0x901237ec88 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=4, OldProtect=0x901237ec88 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=2, OldProtect=0x901237ec88 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=4, OldProtect=0x901237ec88 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=2, OldProtect=0x901237ec88 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=4, OldProtect=0x901237ec88 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=2, OldProtect=0x901237ec88 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=4, OldProtect=0x901237ec88 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ec78 [0x00007ff8e3263000], Size=0x901237ec70 [0x1000], NewProtect=2, OldProtect=0x901237ec88 [4]) => 0

NtOpenKey(KeyHandle=0x901237f260, DesiredAccess=0x3, ObjectAttributes="\Registry\MACHINE\System\CurrentControlSet\Control\SafeBoot\Option") => 0xc0000034 [2 'x ефрхЄё эрщЄш єърчрээ\щ Ърщы.']

NtOpenKey(KeyHandle=0x901237f240, DesiredAccess=KEY_READ, ObjectAttributes="\Registry\Machine\System\CurrentControlSet\Control\Srp\GP\DLL") => 0xc0000034 [2 'x ефрхЄё эрщЄш єърчрээ\щ Ърщы.']

NtOpenKey(KeyHandle=0x901237f238 [0x88], DesiredAccess=0x1, ObjectAttributes="\Registry\Machine\Software\Policies\Microsoft\Windows\Safer\CodeIdentifiers") => 0

NtQueryValueKey(KeyHandle=0x88, ValueName="TransparentEnabled", KeyValueInformationClass=2 [KeyValuePartialInformation], KeyValueInformation=0x901237f2f0, Length=0x50, ResultLength=0x901237f230) => 0xc0000034 [2 'x ефрхЄё эрщЄш єърчрээ\щ Ърщы.']

NtClose(Handle=0x88) => 0

NtQueryInformationToken(TokenHandle=-6, TokenInformationClass=1 [TokenUser], TokenInformation=0x901237f180, Length=0x58, ReturnLength=0x901237f178 [0x2c]) => 0

NtOpenKey(KeyHandle=0x901237f238, DesiredAccess=0x1, ObjectAttributes="\REGISTRY\USER\S-1-5-21-279210374-529612743-1025975986-1001\Software\Policies\Microsoft\Windows\Safer\CodeIdentifiers") => 0xc0000034 [2 'x ефрхЄё эрщЄш єърчрээ\щ Ърщы.']

NtOpenKey(KeyHandle=0x901237f320 [0x88], DesiredAccess=KEY_READ, ObjectAttributes="\Registry\Machine\System\CurrentControlSet\Control\FileSystem") => 0

NtQueryValueKey(KeyHandle=0x88, ValueName="LongPathsEnabled", KeyValueInformationClass=2 [KeyValuePartialInformation], KeyValueInformation=0x901237f360, Length=0x14, ResultLength=0x901237f328 [0x10]) => 0

NtClose(Handle=0x88) => 0

NtSetInformationProcess(ProcessHandle=-1, ProcessInformationClass=0x5d [ProcessLoaderDetour], ProcessInformation=0x901237f300, Length=4) => 0

NtQuerySystemInformationEx(SystemInformationClass=0x6b [SystemLogicalProcessorAndGroupInformation], QueryType=0x901237f280 [1], Alignment=4, SystemInformation=0x260596c3140, Length=0x50, ReturnLength=0x901237f278 [0x30]) => 0

NtCreateIoCompletion(IoHandle=0x260596c6210 [0x88], DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|SYNCHRONIZE|0x3, ObjectAttributes=null, NumberOfConcurrentThreads=5) => 0

NtCreateWorkerFactory(WorkerFactoryHandleReturn=0x260596c6208 [0x8c], DesiredAccess=DELETE|READ_CONTROL|WRITE_DAC|WRITE_OWNER|0xff, ObjectAttributes=null, CompletionPortHandle=0x88, WorkerProcessHandle=-1, StartRoutine=0x7ff8e54bd110, StartParameter=0x260596c61d0, MaxThreadCount=0x200, StackReserve=0x00100000, StackCommit=0x1000) => 0

NtSetInformationWorkerFactory(WorkerFactoryHandle=0x8c, InformationClass=0xd [WorkerFactoryFlags], Buffer=0x901237f348, BufferLength=4) => 0

NtCreateTimer2(TimerHandle=0x260596c6260 [0x90], Unknown1=null, ObjectAttributes=null, Attributes=8, DesiredAccess=SYNCHRONIZE|0x2) => 0

NtCreateWaitCompletionPacket(WaitCompletionPacketHandle=0x260596c6268 [0x94], DesiredAccess=0x1, ObjectAttributes=null) => 0

NtAssociateWaitCompletionPacket(WaitCompletionPacketHandle=0x94, IoCompletionHandle=0x88, TargetObjectHandle=0x90, KeyContext=0x260596c6270, ApcContext=0x260596c6240, IoStatus=0, IoStatusInformation=1, AlreadySignaled=0x901237f240 [0]) => 0

NtCreateTimer2(TimerHandle=0x260596c62d8 [0x98], Unknown1=null, ObjectAttributes=null, Attributes=8, DesiredAccess=SYNCHRONIZE|0x26000000002) => 0

NtCreateWaitCompletionPacket(WaitCompletionPacketHandle=0x260596c62e0 [0x9c], DesiredAccess=0x1, ObjectAttributes=null) => 0

NtAssociateWaitCompletionPacket(WaitCompletionPacketHandle=0x9c, IoCompletionHandle=0x88, TargetObjectHandle=0x98, KeyContext=0x260596c62e8, ApcContext=0x260596c6240, IoStatus=0, IoStatusInformation=0, AlreadySignaled=0x901237f240 [0]) => 0

NtSetInformationWorkerFactory(WorkerFactoryHandle=0x8c, InformationClass=2 [WorkerFactoryIdleTimeout], Buffer=0x901237f348, BufferLength=8) => 0

NtSetInformationWorkerFactory(WorkerFactoryHandle=0x8c, InformationClass=5 [WorkerFactoryThreadMaximum], Buffer=0x901237f348, BufferLength=4) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f350 [0x00007ff6a5a13000], Size=0x901237f358 [0x1000], NewProtect=4, OldProtect=0x901237f6e8 [2]) => 0

NtOpenSection(SectionHandle=0x901237ede8, DesiredAccess=0xd,
ObjectAttributes=0x3c:"VCRUNTIME140D.dll") => 0xc0000034 [2 '═х єфрхЄё эрщЄш єърчрээ√щ
Їрщы.']

NtOpenSection(SectionHandle=0x901237ede8, DesiredAccess=0xd,
ObjectAttributes=0x3c:"ucrtbased.dll") => 0xc0000034 [2 '═х єфрхЄё эрщЄш єърчрээ√щ Їрщы.']

NtQueryWnfStateData(StateName=0x901237ebe0 [0xa3bc7c75], TypeId=null, ExplicitScope=null,
ChangeStamp=0x901237ec78 [0], Buffer=0x901237ecd0, BufferSize=0x901237ebd0 [0]) => 0

NtQueryWnfStateData(StateName=0x901237ea80 [0xa3bc7c75], TypeId=null, ExplicitScope=null,
ChangeStamp=0x901237eb18 [0], Buffer=0x901237eb70, BufferSize=0x901237ea70 [0]) => 0

NtOpenKey(KeyHandle=0x901237ee50 [0xa0], DesiredAccess=0x1,
ObjectAttributes="\Registry\MACHINE\System\CurrentControlSet\Control\Session Manager") => 0

NtQueryValueKey(KeyHandle=0xa0, ValueName="SafeDllSearchMode", KeyValueInformationClass=2
[KeyValuePartialInformation], KeyValueInformation=0x901237ee60, Length=0x10,
ResultLength=0x901237ee58) => 0xc0000034 [2 '═х єфрхЄё эрщЄш єърчрээ√щ Їрщы.']

NtAllocateVirtualMemory(ProcessHandle=-1, IpAddress=0x901237e630 [0x00000260596c7000],
ZeroBits=0, pSize=0x901237e6d8 [0x1000], flAllocationType=0x1000, flProtect=4) => 0

NtQueryAttributesFile(ObjectAttributes="???C:\Users\T470\source\repos\lab44\x64\Debug\VCRUNTIM
E140D.dll", Attributes=0x901237eef8) => 0xc0000034 [2 '═х єфрхЄё эрщЄш єърчрээ√щ Їрщы.']

NtQueryAttributesFile(ObjectAttributes="???C:\Windows\SYSTEM32\VCRUNTIME140D.dll",
Attributes=0x901237eef8 [ARCHIVE]) => 0

NtOpenFile(FileHandle=0x901237f000 [0xa4], DesiredAccess=SYNCHRONIZE|0x21,
ObjectAttributes="???C:\Windows\SYSTEM32\VCRUNTIME140D.dll", IoStatusBlock=0x901237f068
[0/1], ShareAccess=5, OpenOptions=0x60) => 0

NtCreateSection(SectionHandle=0x901237f008 [0xa8], DesiredAccess=0xd, ObjectAttributes=null,
SectionSize=null, Protect=0x10, Attributes=0x01000000, FileHandle=0xa4) => 0

Loaded DLL at 00007FF8B5250000 C:\Windows\SYSTEM32\VCRUNTIME140D.dll

NtMapViewOfSection(SectionHandle=0xa8, ProcessHandle=-1, BaseAddress=0x260596c6410
[0x00007ff8b5250000], ZeroBits=0, CommitSize=0, SectionOffset=null, ViewSize=0x260596c36e0
[0x0002b000], InheritDisposition=1 [ViewShare], AllocationType=0x00800000, Protect=0x80) => 0

NtQueryPerformanceCounter(Counter=0x901237eea0 [8.72141e+12], Freq=null) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237eeb0 [0x00007ff8e55f1000],
Size=0x901237eea8 [0x4000], NewProtect=4, OldProtect=0x901237eea0 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237eeb0 [0x00007ff8e55f1000],
Size=0x901237eea8 [0x4000], NewProtect=2, OldProtect=0x901237eea0 [4]) => 0

NtQuerySecurityAttributesToken(TokenHandle=-4, Attributes="WIN://SYSAPPID",
NumberOfAttributes=2, Buffer=0x901237e980, Length=0x330, ReturnLength=0x901237e938) =>
0xc0000225 [1168 '■ ыхххэЄ эх эрщфхэ.']

NtQuerySecurityAttributesToken(TokenHandle=-4, Attributes="WIN://SYSAPPID",
NumberOfAttributes=1, Buffer=0x901237e980, Length=0x330, ReturnLength=0x901237e938) =>
0xc0000225 [1168 ҮХХЭС эх эрцфхэ.]

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ef00 [0x00007ff8b5270000],
Size=0x901237ef08 [0x1000], NewProtect=4, OldProtect=0x260596c36c8 [2]) => 0

NtOpenSection(SectionHandle=0x901237e998, DesiredAccess=0xd,
ObjectAttributes=0x3c:"ucrtbased.dll") => 0xc0000034 [2 'х фрхЄ эрцЄш єрчрээ\ш їршы.']

NtClose(Handle=0xa8) => 0

NtClose(Handle=0xa4) => 0

NtQueryAttributesFile(ObjectAttributes="??\C:\Users\T470\source\repos\lab44\x64\Debug\ucrtbased.dll",
, Attributes=0x901237eef8) => 0xc0000034 [2 'х фрхЄ эрцЄш єрчрээ\ш їршы.']

NtQueryAttributesFile(ObjectAttributes="??\C:\Windows\SYSTEM32\ucrtbased.dll",
Attributes=0x901237eef8 [ARCHIVE]) => 0

NtOpenFile(FileHandle=0x901237f000 [0xa8], DesiredAccess=SYNCHRONIZE|0x21,
ObjectAttributes="??\C:\Windows\SYSTEM32\ucrtbased.dll", IoStatusBlock=0x901237f068 [0/1],
ShareAccess=5, OpenOptions=0x60) => 0

NtCreateSection(SectionHandle=0x901237f008 [0xa4], DesiredAccess=0xd, ObjectAttributes=null,
SectionSize=null, Protect=0x10, Attributes=0x01000000, FileHandle=0xa8) => 0

Loaded DLL at 00007FF8B45C0000 C:\Windows\SYSTEM32\ucrtbased.dll

NtMapViewOfSection(SectionHandle=0xa4, ProcessHandle=-1, BaseAddress=0x260596c6670
[0x00007ff8b45c0000], ZeroBits=0, CommitSize=0, SectionOffset=null, ViewSize=0x260596c65d0
[0x00221000], InheritDisposition=1 [ViewShare], AllocationType=0x00800000, Protect=0x80) => 0

NtQueryPerformanceCounter(Counter=0x901237eea0 [8.72142e+12], Freq=null) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237eeb0 [0x00007ff8e55f1000],
Size=0x901237eea8 [0x4000], NewProtect=4, OldProtect=0x901237eea0 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237eeb0 [0x00007ff8e55f1000],
Size=0x901237eea8 [0x4000], NewProtect=2, OldProtect=0x901237eea0 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ef00 [0x00007ff8b476b000],
Size=0x901237ef08 [0x1000], NewProtect=4, OldProtect=0x260596c65b8 [2]) => 0

NtClose(Handle=0xa4) => 0

NtClose(Handle=0xa8) => 0

NtQueryAttributesFile(ObjectAttributes="??\C:\Users\T470\source\repos\lab44\x64\Debug\ucrtbased.dll",
, Attributes=0x901237eef8) => 0xc0000034 [2 'х фрхЄ эрцЄш єрчрээ\ш їршы.']

NtQueryAttributesFile(ObjectAttributes="??\C:\Windows\SYSTEM32\ucrtbased.dll",
Attributes=0x901237eef8 [ARCHIVE]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f6c8 [0x00007ff6a5a13000], Size=0x901237f6d0 [0x1000], NewProtect=2, OldProtect=0x901237f220 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x260596c6598 [0x00007ff8b476b000], Size=0x260596c65a0 [0x1000], NewProtect=2, OldProtect=0x901237f220 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x260596c36a8 [0x00007ff8b5270000], Size=0x260596c36b0 [0x1000], NewProtect=2, OldProtect=0x901237f220 [4]) => 0

NtApphelpCacheControl(ServiceClass=6, ServiceData="") => 0

NtApphelpCacheControl(ServiceClass=6, ServiceData="") => 0

NtApphelpCacheControl(ServiceClass=6, ServiceData="") => 0

NtSetInformationVirtualMemory(ProcessHandle=-1, VmInformationClass=4, NumberOfEntries=1, VirtualAddresses=0x901237f588, VmInformation=0x901237f4b0, VmInformationLength=4) => 0xc00000bb [50 'Пррющ чряЁюё эх яюфхЁцштрхЄё.']

NtQueryInformationThread(ThreadHandle=-2, ThreadInformationClass=0x11 [ThreadHideFromDebugger], ThreadInformation=0x901237f3c0, Length=1, ReturnLength=null) => 0

Initial breakpoint

NtSetEvent(EventHandle=0x44, PrevState=null) => 0

NtSetInformationVirtualMemory(ProcessHandle=-1, VmInformationClass=4, NumberOfEntries=1, VirtualAddresses=0x901237f240, VmInformation=0x901237f318, VmInformationLength=4) => 0xc00000bb [50 'Пррющ чряЁюё эх яюфхЁцштрхЄё.']

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237e610 [0x00000260596c8000], ZeroBits=0, pSize=0x901237e6b8 [0x2000], flAllocationType=0x1000, flProtect=4) => 0

NtQueryVolumeInformationFile(FileHandle=0x54, IoStatusBlock=0x901237ef60 [0/8], FsInformation=0x901237ef80, Length=8, FsInformationClass=4 [FsDeviceInformation]) => 0

NtQueryVolumeInformationFile(FileHandle=0x58, IoStatusBlock=0x901237ef60 [0/8], FsInformation=0x901237ef80, Length=8, FsInformationClass=4 [FsDeviceInformation]) => 0

NtQueryVolumeInformationFile(FileHandle=0x5c, IoStatusBlock=0x901237ef60 [0/8], FsInformation=0x901237ef80, Length=8, FsInformationClass=4 [FsDeviceInformation]) => 0

NtOpenKey(KeyHandle=0x901237da40 [0xa4], DesiredAccess=KEY_READ, ObjectAttributes="\Registry\Machine\System\CurrentControlSet\Control\Nls\Sorting\Versions") => 0

NtQueryValueKey(KeyHandle=0xa4, ValueName="", KeyValueInformationClass=1 [KeyValueFullInformation], KeyValueInformation=0x901237df20, Length=0x214, ResultLength=0x901237ded8 [0x2a]) => 0

NtQueryValueKey(KeyHandle=0xa4, ValueName="000603xx", KeyValueInformationClass=1 [KeyValueFullInformation], KeyValueInformation=0x901237df00, Length=0x214, ResultLength=0x901237dcb8 [0x42]) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237e670 [0x00000260596ca000], ZeroBits=0, pSize=0x901237e718 [0x2000], flAllocationType=0x1000, flProtect=4) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237e650 [0x00000260596cc000], ZeroBits=0, pSize=0x901237e6f8 [0x2000], flAllocationType=0x1000, flProtect=4) => 0

NtSetInformationVirtualMemory(ProcessHandle=-1, VmInformationClass=4, NumberOfEntries=1, VirtualAddresses=0x901237f280, VmInformation=0x901237f358, VmInformationLength=4) => 0xc00000bb [50 'Прыюц чряЁюё эх яюфхЁцшрхЄё .']

NtSetEvent(EventHandle=0x44, PrevState=null) => 0

NtSetInformationWorkerFactory(WorkerFactoryHandle=0x8c, InformationClass=3 [WorkerFactoryBindingCount], Buffer=0x901237f6e8, BufferLength=4) => 0

NtSetEvent(EventHandle=8, PrevState=null) => 0

NtTestAlert() => 0

NtQueryVirtualMemory(ProcessHandle=-1, BaseAddress=0x7ff6a5a01122, MemoryInformationClass=0 [MemoryBasicInformation], MemoryInformation=0x901237f900, Length=0x30, ReturnLength=0x901237f8b0 [0x30]) => 0

NtQueryVirtualMemory(ProcessHandle=-1, BaseAddress=0x7ff6a5a01122, MemoryInformationClass=3 [MemoryRegionInformation], MemoryInformation=0x901237f930, Length=0x30, ReturnLength=null) => 0

NtQueryVirtualMemory(ProcessHandle=-1, BaseAddress=0x7ff6a5a01122, MemoryInformationClass=2 [MemoryMappedFilenameInformation], MemoryInformation=0x901237f9a8, Length=0x21a, ReturnLength=null) => 0

Could not load the DLL, using system funcs.

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237ea90 [0/0x2d], IoControlCode=0x00500016, InputBuffer=0x901237eaa0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f6b0 [0x0000026059670000], ZeroBits=0, pSize=0x901237f6b8 [0x1000], flAllocationType=0x3000, flProtect=4) => 0

System allocator was created

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x1d], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System alloc 10 bytes

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x16], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b0 [0x0000026059680000], ZeroBits=0, pSize=0x901237f5b8 [0x1000], flAllocationType=0x3000, flProtect=4) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System alloc 10 bytes

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x16], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b0 [0x0000026059690000], ZeroBits=0, pSize=0x901237f5b8 [0x1000], flAllocationType=0x3000, flProtect=4) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System alloc 10 bytes

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x16], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b0 [0x00000260596a0000], ZeroBits=0, pSize=0x901237f5b8 [0x1000], flAllocationType=0x3000, flProtect=4) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System alloc 10 bytes

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x16], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b0 [0x00000260596b0000], ZeroBits=0, pSize=0x901237f5b8 [0x1000], flAllocationType=0x3000, flProtect=4) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System alloc 10 bytes

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x16], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b0 [0x0000026059890000], ZeroBits=0, pSize=0x901237f5b8 [0x1000], flAllocationType=0x3000, flProtect=4) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System alloc 10 bytes

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x16], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b0 [0x00000260598a0000], ZeroBits=0, pSize=0x901237f5b8 [0x1000], flAllocationType=0x3000, flProtect=4) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System alloc 10 bytes

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x16], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b0 [0x00000260598b0000], ZeroBits=0, pSize=0x901237f5b8 [0x1000], flAllocationType=0x3000, flProtect=4) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System alloc 10 bytes

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x16], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b0 [0x00000260598c0000], ZeroBits=0, pSize=0x901237f5b8 [0x1000], flAllocationType=0x3000, flProtect=4) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System alloc 10 bytes

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x16], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b0 [0x00000260598d0000], ZeroBits=0, pSize=0x901237f5b8 [0x1000], flAllocationType=0x3000, flProtect=4) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System alloc 10 bytes

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x16], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b0 [0x00000260598e0000], ZeroBits=0, pSize=0x901237f5b8 [0x1000], flAllocationType=0x3000, flProtect=4) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

Allocation time: 0.27600000 seconds

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237ea90 [0/0x24], IoControlCode=0x00500016, InputBuffer=0x901237eaa0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System free

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0xc], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b8 [0x0000026059680000], pSize=0x901237f5b0 [0x1000], flFreeType=0x8000) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System free

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0xc], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b8 [0x0000026059690000], pSize=0x901237f5b0 [0x1000], flFreeType=0x8000) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System free

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0xc], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b8 [0x00000260596a0000], pSize=0x901237f5b0 [0x1000], flFreeType=0x8000) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System free

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0xc], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b8 [0x00000260596b0000], pSize=0x901237f5b0 [0x1000], flFreeType=0x8000) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System free

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0xc], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b8 [0x0000026059890000], pSize=0x901237f5b0 [0x1000], flFreeType=0x8000) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System free

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0xc], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b8 [0x00000260598a0000], pSize=0x901237f5b0 [0x1000], flFreeType=0x8000) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System free

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0xc], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b8 [0x00000260598b0000], pSize=0x901237f5b0 [0x1000], flFreeType=0x8000) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System free

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0xc], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b8 [0x00000260598c0000], pSize=0x901237f5b0 [0x1000], flFreeType=0x8000) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System free

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0xc], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b8 [0x00000260598d0000], pSize=0x901237f5b0 [0x1000], flFreeType=0x8000) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System free

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0xc], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f5b8 [0x00000260598e0000], pSize=0x901237f5b0 [0x1000], flFreeType=0x8000) => 0

System print blocks

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x14], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

Free time: 0.16400000 seconds

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237ea90 [0/0x1e], IoControlCode=0x00500016, InputBuffer=0x901237eaa0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

System allocator was destroyed

NtDeviceIoControlFile(FileHandle=0x58, Event=0, ApcRoutine=null, ApcContext=null, IoStatusBlock=0x901237e990 [0/0x1f], IoControlCode=0x00500016, InputBuffer=0x901237e9a0, InputBufferLength=0x40, OutputBuffer=null, OutputBufferLength=0) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237f6b8 [0x0000026059670000], pSize=0x901237f6b0 [0x1000], flFreeType=0x8000) => 0

NtSetInformationProcess(ProcessHandle=-1, ProcessInformationClass=0x5d [ProcessLoaderDetour], ProcessInformation=0x901237f520, Length=4) => 0

NtOpenSection(SectionHandle=0x901237f4f8, DesiredAccess=0xd, ObjectAttributes=0x3c:"kernel.appcore.dll") => 0xc0000034 [2 'мх єфрхЄё эрщЄш єърчрээ√щ Ырщы.']

NtQueryAttributesFile(ObjectAttributes="\\?\\C:\\Windows\\SYSTEM32\\kernel.appcore.dll", Attributes=0x901237f2b8 [ARCHIVE]) => 0

NtOpenFile(FileHandle=0x901237f2b0 [0xa8], DesiredAccess=SYNCHRONIZE|0x21, ObjectAttributes="\??\C:\Windows\SYSTEM32\kernel.appcore.dll", IoStatusBlock=0x901237f318 [0/1], ShareAccess=5, OpenOptions=0x60) => 0

NtCreateSection(SectionHandle=0x901237f2b8 [0xac], DesiredAccess=0xd, ObjectAttributes=null, SectionSize=null, Protect=0x10, Attributes=0x01000000, FileHandle=0xa8) => 0

Loaded DLL at 00007FF8E0950000 C:\Windows\SYSTEM32\kernel.appcore.dll

NtMapViewOfSection(SectionHandle=0xac, ProcessHandle=-1, BaseAddress=0x260596caa80 [0x00007ff8e0950000], ZeroBits=0, CommitSize=0, SectionOffset=null, ViewSize=0x260596c6c20 [0x00012000], InheritDisposition=1 [ViewShare], AllocationType=0x00800000, Protect=0x80) => 0

NtQueryPerformanceCounter(Counter=0x901237f150 [8.72142e+12], Freq=null) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f168 [0x00007ff8e095f000], Size=0x901237f160 [0x1000], NewProtect=2, OldProtect=0x901237f1d0 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f160 [0x00007ff8e55f1000], Size=0x901237f158 [0x4000], NewProtect=4, OldProtect=0x901237f150 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f160 [0x00007ff8e55f1000], Size=0x901237f158 [0x4000], NewProtect=2, OldProtect=0x901237f150 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237f1b0 [0x00007ff8e0955000], Size=0x901237f1b8 [0x1000], NewProtect=4, OldProtect=0x260596c6c08 [2]) => 0

NtOpenSection(SectionHandle=0x901237ec48 [0xb0], DesiredAccess=0xd, ObjectAttributes=0x3c:"msvcrt.dll") => 0

Loaded DLL at 00007FF8E42A0000 C:\Windows\System32\msvcrt.dll

NtMapViewOfSection(SectionHandle=0xb0, ProcessHandle=-1, BaseAddress=0x260596cc9c0 [0x00007ff8e42a0000], ZeroBits=0, CommitSize=0, SectionOffset=null, ViewSize=0x260596cc920 [0x0009e000], InheritDisposition=1 [ViewShare], AllocationType=0x00800000, Protect=0x80) => 0

NtQueryPerformanceCounter(Counter=0x901237eac0 [8.72142e+12], Freq=null) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ead0 [0x00007ff8e55f1000], Size=0x901237eac8 [0x4000], NewProtect=4, OldProtect=0x901237eac0 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ead0 [0x00007ff8e55f1000], Size=0x901237eac8 [0x4000], NewProtect=2, OldProtect=0x901237eac0 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237eb20 [0x00007ff8e4316000], Size=0x901237eb28 [0x2000], NewProtect=4, OldProtect=0x260596cc908 [2]) => 0

NtClose(Handle=0xb0) => 0

NtOpenSection(SectionHandle=0x901237ec48 [0xb0], DesiredAccess=0xd, ObjectAttributes=0x3c:"RPCRT4.dll") => 0

Loaded DLL at 00007FF8E3820000 C:\Windows\System32\RPCRT4.dll

NtMapViewOfSection(SectionHandle=0xb0, ProcessHandle=-1, BaseAddress=0x260596ccd10 [0x00007ff8e3820000], ZeroBits=0, CommitSize=0, SectionOffset=null, ViewSize=0x260596ccc70 [0x00123000], InheritDisposition=1 [ViewShare], AllocationType=0x00800000, Protect=0x80) => 0

NtQueryPerformanceCounter(Counter=0x901237eac0 [8.72142e+12], Freq=null) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ead8 [0x00007ff8e393b000], Size=0x901237ead0 [0x1000], NewProtect=2, OldProtect=0x901237eb40 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ead0 [0x00007ff8e55f1000], Size=0x901237eac8 [0x4000], NewProtect=4, OldProtect=0x901237eac0 [2]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237ead0 [0x00007ff8e55f1000], Size=0x901237eac8 [0x4000], NewProtect=2, OldProtect=0x901237eac0 [4]) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x901237eb20 [0x00007ff8e390c000], Size=0x901237eb28 [0x2000], NewProtect=4, OldProtect=0x260596ccc58 [2]) => 0

NtClose(Handle=0xb0) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x260596c6be8 [0x00007ff8e0955000], Size=0x260596c6bf0 [0x1000], NewProtect=2, OldProtect=0x901237f0a0 [4]) => 0

NtClose(Handle=0xac) => 0

NtClose(Handle=0xa8) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x260596cc8e8 [0x00007ff8e4316000], Size=0x260596cc8f0 [0x2000], NewProtect=2, OldProtect=0x901237f3c0 [4]) => 0

NtSetInformationProcess(ProcessHandle=-1, ProcessInformationClass=0x23 [ProcessTlsInformation], ProcessInformation=0x901237f320, Length=0x28) => 0

NtProtectVirtualMemory(ProcessHandle=-1, BaseAddress=0x260596ccc38 [0x00007ff8e390c000], Size=0x260596ccc40 [0x2000], NewProtect=2, OldProtect=0x901237f3c0 [4]) => 0

NtApphelpCacheControl(ServiceClass=6, ServiceData='') => 0

NtApphelpCacheControl(ServiceClass=6, ServiceData='') => 0

NtApphelpCacheControl(ServiceClass=6, ServiceData='') => 0

NtSetInformationVirtualMemory(ProcessHandle=-1, VmInformationClass=4, NumberOfEntries=1, VirtualAddresses=0x901237f3e0, VmInformation=0x901237f4b8, VmInformationLength=4) => 0xc00000bb [50 'пррющ чряЁюё эх яюффхЁцштрхЄё .']

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237ec10 [0x0000026059890000], ZeroBits=0, pSize=0x901237ec18 [0x000a0000], flAllocationType=0x2000, flProtect=4) => 0

NtFreeVirtualMemory(ProcessHandle=-1, lpAddress=0x901237ec10 [0x0000026059890000], pSize=0x901237ec08 [0x00090000], flFreeType=0x8000) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, lpAddress=0x901237ebf8 [0x0000026059920000], ZeroBits=0, pSize=0x901237ebf0 [0x2000], flAllocationType=0x1000, flProtect=4) => 0

```

NtQueryVolumeInformationFile(FileHandle=0x54, IoStatusBlock=0x901237f1e0 [0/8],
FsInformation=0x901237f200, Length=8, FsInformationClass=4 [FsDeviceInformation]) => 0

NtQueryVolumeInformationFile(FileHandle=0x58, IoStatusBlock=0x901237f1e0 [0/8],
FsInformation=0x901237f200, Length=8, FsInformationClass=4 [FsDeviceInformation]) => 0

NtQueryVolumeInformationFile(FileHandle=0x5c, IoStatusBlock=0x901237f1e0 [0/8],
FsInformation=0x901237f200, Length=8, FsInformationClass=4 [FsDeviceInformation]) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, IpAddress=0x901237ea80 [0x00000260596ce000],
ZeroBits=0, pSize=0x901237eb28 [0x2000], flAllocationType=0x1000, flProtect=4) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, IpAddress=0x901237ea50 [0x0000026059922000],
ZeroBits=0, pSize=0x901237eaf8 [0x1000], flAllocationType=0x1000, flProtect=4) => 0

NtAllocateVirtualMemory(ProcessHandle=-1, IpAddress=0x901237ea30 [0x0000026059923000],
ZeroBits=0, pSize=0x901237ead8 [0x2000], flAllocationType=0x1000, flProtect=4) => 0

NtSetInformationVirtualMemory(ProcessHandle=-1, VmInformationClass=4, NumberOfEntries=1,
VirtualAddresses=0x901237f3e0, VmInformation=0x901237f4b8, VmInformationLength=4) =>
0xc00000bb [50 'тґрґющ чґрґёюё эх яюфґхёцшґрґёё .']

NtSetInformationVirtualMemory(ProcessHandle=-1, VmInformationClass=4, NumberOfEntries=1,
VirtualAddresses=0x901237f420, VmInformation=0x901237f4f8, VmInformationLength=4) =>
0xc00000bb [50 'тґрґющ чґрґёюё эх яюфґхёцшґрґёё .']

NtSetEvent(EventHandle=0x44, PrevState=null) => 0

NtQuerySecurityAttributesToken(TokenHandle=-6, Attributes="WIN://SYSAPPID",
NumberOfAttributes=2, Buffer=0x901237f650, Length=0x330, ReturnLength=0x901237f608) =>
0xc0000225 [1168 'ґыхьхэё эх эрщґхэ.']

NtQuerySecurityAttributesToken(TokenHandle=-6, Attributes="WIN://SYSAPPID",
NumberOfAttributes=1, Buffer=0x901237f650, Length=0x330, ReturnLength=0x901237f608) =>
0xc0000225 [1168 'ґыхьхэё эх эрщґхэ.']

NtTerminateProcess(ProcessHandle=0, ExitStatus=0) => 0

NtClose(Handle=0x80) => 0

NtClose(Handle=0x50) => 0

NtClose(Handle=0x7c) => 0

NtClose(Handle=0x84) => 0

NtQueryWnfStateData(StateName=0x901237f700 [0xa3bc1c75], TypeId=null, ExplicitScope=null,
ChangeStamp=0x901237e644 [0x6caa], Buffer=0x901237e6a0, BufferSize=0x901237e640 [0xb04]) => 0

NtQueryWnfStateData(StateName=0x901237f588 [0xa3bc7c75], TypeId=null, ExplicitScope=null,
ChangeStamp=0x901237f618 [0], Buffer=0x901237f670, BufferSize=0x901237f570 [0]) => 0

Process 5596 exit code: 0

```

Вывод

В ходе выполнения лабораторной работы я разработал и отладил программу на языке Си, которая загружает динамическую библиотеку по переданному в аргументах командной строки пути. Помимо этого я исследовал два алгоритма аллокации памяти и создал динамические библиотеки, реализующие их.